

A
Major Project Report on
**Investigraph: A Multi-Strategy GraphRAG
Framework for Natural Language Querying of
POLE Crime Investigation Knowledge Graphs**

submitted in partial fulfillment of the requirements
for the award of the degree of

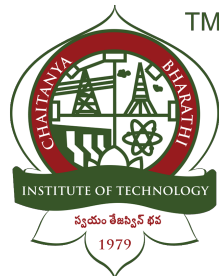
Bachelor of Engineering
in
Artificial Intelligence and Data Science

By
Valleboina Shiva Prasad (1601-22-771-063)
Sangani Guna Sahithi (1601-22-771-020)
Kolluri Venkata Sai Rahul (1601-22-771-049)

Under the esteemed guidance of

Ms. Yenigalla Pallavi

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
HYDERABAD – 500075

APRIL 2026



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
HYDERABAD – 500075**

INSTITUTE VISION

“To be the center of excellence in technical education and research”.

INSTITUTE MISSION

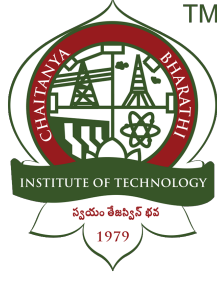
“To address the emerging needs through quality technical education and advanced research”.

DEPARTMENT VISION

”To be a globally recognized center of excellence in the field of Artificial Intelligence and Data Science that produces innovative pioneers and research experts capable of addressing complex real-world challenges and contributing to the socio-economic development of the nation.”

DEPARTMENT MISSION

1. To provide cutting-edge education in the field of Artificial Intelligence and Data Science that is rooted in ethical and moral values.
2. To establish strong partnerships with industries and research organizations in the field of Artificial Intelligence and Data Science, and to excel in the emerging areas of research by creating innovative solutions.
3. To cultivate a strong sense of social responsibility among students, fostering their inclination to utilize their knowledge and skills for the betterment of society.
4. To motivate and mentor students to become trailblazers in Artificial Intelligence and Data Science, and develop an entrepreneurial mindset that nurtures innovation and creativity.



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
HYDERABAD – 500075**

DECLARATION CERTIFICATE

We hereby declare that the project titled **Investigraph: A Multi-Strategy GraphRAG Framework for Natural Language Querying of POLE Crime Investigation Knowledge Graphs** submitted by us to the **Artificial Intelligence and Data Science CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY, HYDERABAD** in partial fulfillment of the requirements for the award of **Bachelor of Engineering** is a bona-fide record of the work carried out by us under the supervision of **Ms. Yenigalla Pallavi** . We further declare that the work reported in this project, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Project Associates

Valleboina Shiva Prasad (1601-22-771-063)

Sangani Guna Sahithi (1601-22-771-020)

Kolluri Venkata Sai Rahul (1601-22-771-049)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
HYDERABAD – 500075
BONAFIDE CERTIFICATE

This is to certify that the project titled **Investigraph: A Multi-Strategy GraphRAG Framework for Natural Language Querying of POLE Crime Investigation Knowledge Graphs** is a bonafide record of the work done by

Valleboina Shiva Prasad (1601-22-771-063)

Sangani Guna Sahithi (1601-22-771-020)

Kolluri Venkata Sai Rahul (1601-22-771-049)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Data Science** to the **CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY, HYDERABAD** carried out under my guidance and supervision during the year 2025-26. The results presented in this project report have not been submitted to any other university or Institute for the award of any degree.

Ms. Yenigalla Pallavi

Guide

Dr. K. Radhika

Head of the Department

Submitted for Semester Major-Project viva-voce examination held on _____

Examiner-1

Examiner-2

ABSTRACT

Crime investigation workflows generate vast volumes of interconnected data spanning persons of interest, physical evidence, geographic locations, and criminal events. Traditionally, law enforcement analysts query this data using specialized graph query languages such as Cypher, which presents a steep learning curve and impedes rapid intelligence extraction. Existing natural language interfaces for graph databases typically rely on a single retrieval strategy — often a direct translation of text to structured queries — and are prone to brittle failures when queries contain syntactic errors or yield empty results, limiting their practical utility in time-sensitive investigative scenarios. This project presents **Investigraph**, a production-ready GraphRAG (Graph Retrieval-Augmented Generation) system built on a Neo4j knowledge graph organized around the POLE (Person, Object, Location, Event) data model, which is widely adopted in intelligence-led policing. Our framework introduces a multi-strategy retrieval architecture comprising three complementary approaches: (i) *Text2Cypher*, which translates investigator questions into Cypher queries using schema-aware prompting with 40 curated few-shot examples; (ii) *Vector Retrieval*, which performs semantic search on 384-dimensional Sentence-Transformer embeddings of crime records stored in a native Neo4j vector index; and (iii) *VectorCypher*, a hybrid strategy that combines semantic search with multi-hop graph traversal to capture rich contextual neighborhoods. A heuristic-based question classifier routes each incoming query to the most appropriate retriever, and a self-healing retry mechanism automatically corrects syntax errors and reformulates queries that return empty results, supporting up to three successive attempts per query. Answer generation is handled by the Groq-hosted Llama-3.3-70b-versatile model through the `neo4j-graphrag` orchestration framework, which grounds every response strictly in the retrieved graph context to prevent hallucination. The frontend, built with React 18, TypeScript, and the `vis-network` library, provides interactive graph visualizations alongside natural language answers. Experimental

evaluation demonstrates average end-to-end response times of 1–3 seconds, a query success rate exceeding 95% when self-healing retries are included, and first-attempt accuracy between 85–95% across structured, semantic, and hybrid question types. The system handles 11 node types and 17 relationship types encompassing persons, crimes, officers, vehicles, evidence, communication records, and geographic areas.

Keywords : GraphRAG, Knowledge Graphs, Neo4j, POLE Data Model, Natural Language to Cypher, Retrieval-Augmented Generation, Vector Search, Crime Investigation, Graph Visualization, Self-Healing Queries

ACKNOWLEDGEMENTS

We would like to express our deepest gratitude to the following people for guiding us through this course and without whom this project and the results achieved from it would not have reached completion.

Ms. Yenigalla Pallavi, Assistant Professor, Department of Artificial Intelligence and Data Science, for helping us and guiding us in the course of this project. Without her guidance, we would not have been able to successfully complete this project. Her patience and genial attitude is and always will be a source of inspiration to us.

Dr. K. Radhika, the Head of the Department, Department of Artificial Intelligence and Data Science, for allowing us to avail the facilities at the department.

We show gratitude to our honorable Principal **Prof. C.V. Narsimhulu** for providing all facilities and support. We are also thankful to the faculty and staff members of the Department of Artificial Intelligence and Data Science, our individual parents and our friends for their constant support and help.

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	i
ACKNOWLEDGEMENTS	iii
TABLE OF CONTENTS	iv
LIST OF TABLES	viii
LIST OF FIGURES	ix
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
1.5 Motivation	4
1.6 Existing System	5
1.6.1 Traditional Crime Information Systems	5
1.6.2 Limitations of Existing Systems	5
CHAPTER 2 LITERATURE REVIEW	7
2.1 Knowledge Graphs and Graph Databases for Investigation	7
2.2 Natural Language Interfaces for Graph Databases	8
2.3 Large Language Models for Code and Query Generation	9
2.4 Retrieval-Augmented Generation	10
2.5 GraphRAG: Graph-Enhanced Retrieval	11
2.6 Vector Embeddings and Semantic Search	11
2.7 Summary Table of Literature Survey	13

2.8	Gaps Identified	14
CHAPTER 3 PROPOSED SYSTEM		15
3.1	Overview of the Proposed Architecture	15
3.2	Multi-Strategy Retrieval Framework	16
3.3	Intelligent Query Routing	17
3.4	Prompt Engineering Strategy	17
3.5	Self-Healing Query Execution	18
3.6	Context-Grounded Answer Generation	19
CHAPTER 4 SYSTEM DESIGN AND ARCHITECTURE		21
4.1	Overall System Design	21
4.2	System Architecture	21
4.2.1	Architectural Overview	21
4.2.2	Component Interaction Flow	22
4.2.3	Key Design Principles	23
4.2.4	Component Specifications	24
4.2.5	API Endpoint Design	24
4.3	POLE Knowledge Graph Schema	25
4.3.1	Node Types and Properties	25
4.3.2	Relationship Types	26
4.4	Retrieval Strategy Design	28
4.4.1	Text2Cypher Retriever	28
4.4.2	Vector Retriever	28
4.4.3	VectorCypher Retriever	29
4.5	Fallback and Error Recovery Design	29
CHAPTER 5 SYSTEM IMPLEMENTATION AND EVALUATION		30
5.1	Implementation Overview	30
5.1.1	Schema Introspection Module	30
5.1.2	Few-Shot Example Loader	30
5.1.3	GroqLLM Wrapper	31

5.1.4	Embedding Pipeline	31
5.1.5	Frontend Implementation	31
5.1.6	Deployment Configuration	32
5.2	Testing Framework	33
5.3	Evaluation Results	34
5.3.1	Query Classification Accuracy	34
5.3.2	Retrieval Quality by Strategy	34
5.3.3	Response Time Performance	35
5.3.4	Comparative Evaluation	35
5.4	Challenges and Limitations	36
CHAPTER 6	RESULTS AND DISCUSSIONS	37
6.1	Query Routing Effectiveness	37
6.2	Retrieval Strategy Effectiveness	37
6.3	Self-Healing Success Rate	37
6.4	End-to-End Response Times	38
6.5	Scenario-Based Observations	38
6.6	Key Findings	40
6.7	Practical Applications	40
CHAPTER 7	CONCLUSION	41
CHAPTER 8	FUTURE SCOPE	43
8.1	LLM-Based Query Routing	43
8.2	Multi-Modal Evidence Retrieval	43
8.3	Temporal Crime Pattern Analysis	44
8.4	Advanced Graph Analytics	44
8.5	Security and Privacy Enhancements	44
8.6	Scalability and Real-Time Data Integration	45
APPENDIX A	CODE ATTACHMENTS	46
A.1	GraphRAG Pipeline Orchestrator	46
A.2	Heuristic Question Classifier	48

A.3 Self-Healing Text2Cypher Retriever	50
A.4 VectorCypher Hybrid Retrieval Query	52
A.5 GroqLLM Wrapper	54
A.6 Embedding Pipeline	55
A.7 FastAPI Application Entry Point	56
BIBLIOGRAPHY	58

LIST OF TABLES

2.1	Summary of Key Literature	13
3.1	Comparison of the Three Retrieval Strategies	16
4.1	Technology Stack	21
4.2	Backend Module Specifications	24
4.3	REST API Endpoints	25
4.4	POLE Node Types	26
4.5	POLE Relationship Types	27
5.1	Few-Shot Example Categories	31
5.2	Test Suite Summary	33
5.3	Query Classification Results	34
5.4	Per-Retriever Success Rates	34
5.5	Average Response Times by Module	35
5.6	Comparison with Legacy System	36

LIST OF FIGURES

3.1	High-level architecture of the Investigraph GraphRAG system showing the four-module pipeline from user input to visual output.	15
3.2	State machine diagram of the self-healing Text2Cypher retry mechanism.	19
4.1	Detailed four-layer architecture of Investigraph.	22
4.2	Sequence diagram showing the interaction between system components during query processing.	23
4.3	Visual representation of the POLE schema showing all 11 node types and 17 relationship types.	28

CHAPTER 1

INTRODUCTION

1.1 Introduction

Law enforcement agencies around the world accumulate enormous volumes of interconnected data during the course of criminal investigations. Persons of interest, physical evidence, geographic locations, communication records, and criminal events form a densely linked web of information that analysts must traverse to produce actionable intelligence. Graph databases, particularly those implementing the property graph model, have emerged as a natural fit for representing these relationships because they preserve the structural context that traditional relational databases tend to flatten or lose during normalization [1]. The Neo4j graph database, with its native property graph storage engine and the Cypher query language, has gained widespread adoption in both commercial and government deployments for this very reason [2].

However, the practical value of a knowledge graph is severely limited when the only way to interrogate it requires fluency in a specialized query language. Most investigators are domain experts in criminal intelligence—not database engineers—and expecting them to write multi-hop Cypher traversals under time pressure is unrealistic. Natural language interfaces have been proposed as a solution, but the first generation of these tools tended to rely on a single rigid pipeline: parse the question, generate a single query, execute it, and hope for the best. When the generated query contained even a minor syntax error or targeted a slightly incorrect property name, the entire pipeline would fail silently, returning nothing useful to the analyst at a critical moment [3].

The emergence of large language models (LLMs) and retrieval-augmented generation (RAG) has opened a different path. Rather than relying solely on translating natural language into structured queries, a RAG-based system can combine structured graph traversal with semantic vector search, grounding every generated answer in verified data rather than allowing the model to hallucinate facts [4]. GraphRAG, which enriches this paradigm with graph-structured context, represents the frontier of this line of work [5].

This project presents **Investigraph**, a production-ready GraphRAG system that enables investigators to query a POLE (Person, Object, Location, Event) crime knowledge graph using plain English. The system implements three complemen-

tary retrieval strategies, intelligent query routing, self-healing error correction, and context-grounded answer generation to deliver reliable, transparent, and fast investigation insights.

1.2 Problem Statement

Existing natural language question-answering systems for graph databases suffer from four primary shortcomings that render them inadequate for real-world crime investigation:

1. **Single-strategy retrieval.** Most systems rely exclusively on text-to-query translation (e.g., NL-to-Cypher or NL-to-SPARQL), which is inherently brittle when the question does not map cleanly to a structured query pattern. Semantic or exploratory questions—such as “tell me about incidents involving theft”—require a fundamentally different retrieval approach that keyword-based translation cannot provide [6].
2. **Fragile error handling.** A single syntax error in a generated Cypher query causes the entire pipeline to fail. Legacy systems rarely attempt to diagnose or correct these errors, leaving the investigator with an opaque failure message and no recourse other than rephrasing the question manually.
3. **Context poverty.** Even when a structured query succeeds, the raw results are presented without the surrounding graph context. An investigator who asks about a specific crime receives a flat list of properties but gains no visibility into the connected persons, officers, vehicles, or geographic patterns that constitute the real investigative value of a graph database.
4. **Hallucination risk.** Standard LLM-based approaches that do not ground their responses in retrieved evidence are prone to generating plausible-sounding but factually incorrect answers, which is unacceptable in a forensic or legal context where every claim must be traceable to a verifiable data source.

A unified framework is needed that addresses all four shortcomings: routing questions to specialized retrievers, recovering from query failures, enriching results with graph context, and grounding every answer in the knowledge graph.

1.3 Objectives

The primary objectives of this project are:

1. Design and implement a multi-strategy GraphRAG pipeline with three retrieval modes—Text2Cypher, vector search, and hybrid VectorCypher—to address the full range of investigator queries.
2. Develop a heuristic-based question classifier that routes each incoming question to the most appropriate retrieval strategy based on keyword patterns and query structure.
3. Build a self-healing query execution mechanism that automatically detects and corrects Cypher syntax errors and reformulates queries yielding empty results, supporting up to three retry attempts per query.
4. Implement context-grounded answer generation using the Groq-hosted Llama-3.3-70b-versatile model through the `neo4j-graphrag` framework, ensuring that every response is derived solely from retrieved graph evidence.
5. Create an interactive frontend with React, TypeScript, and `vis-network` that provides natural language answers alongside real-time graph visualizations showing entities and their relationships.
6. Evaluate the system on structured, semantic, and hybrid query types, targeting average response times under 3 seconds and a query success rate above 95% with self-healing retries enabled.

1.4 Scope of the Project

The scope of Investigraph encompasses the design, implementation, and evaluation of a complete GraphRAG pipeline for querying a POLE crime investigation knowledge graph using natural language. The project boundaries are defined as follows:

- **Data domain.** The system operates on a POLE knowledge graph containing 11 node types and 17 relationship types, populated with approximately 28,000 crime records from a synthetic law enforcement dataset. The data represents a realistic crime intelligence scenario including persons of interest, criminal incidents, geographic locations, investigating officers, vehicles, phone records, and their interconnections. The dataset is pre-loaded into Neo4j and is treated as a static snapshot for the purposes of this project.
- **Query coverage.** The system supports three broad classes of investigator questions: structured queries requiring precise counts, filters, or aggregations (e.g., “How many crimes happened in area WN?”); semantic queries involving vague

descriptions or keyword lookups (e.g., “Shoplifting”); and hybrid queries that require both semantic matching and structural graph traversal for contextual depth (e.g., “Tell me about people connected to drug crimes in Bolton”).

- **System boundary.** Investigraph is delivered as a standalone web application with a FastAPI Python backend and a React TypeScript frontend. It connects to a single Neo4j database instance and uses Groq’s hosted LLM API for inference. The system does not include data ingestion pipelines, entity resolution, real-time streaming, or integration with external law enforcement record management systems. It assumes the POLE knowledge graph has already been constructed and populated.
- **Evaluation scope.** The system is evaluated on four dimensions: query classification accuracy (55 test questions), per-retriever success rates (with and without retry/fallback), end-to-end response time performance, and self-healing recovery rates. Four representative investigation scenarios are used to demonstrate qualitative system behavior under realistic conditions.
- **Non-goals.** The project does not address the construction of the POLE knowledge graph from raw police records, the training or fine-tuning of language models, or the deployment of the system within a classified law enforcement network. These topics are identified as future work.

1.5 Motivation

The motivation for this work arises from a convergence of technological maturity and operational need. On the technology side, three capabilities have recently reached a level of reliability that makes the proposed system feasible:

- **Native vector indexing in Neo4j.** Starting with version 5.11, Neo4j supports built-in vector similarity indexes alongside its graph engine, eliminating the need for a separate vector database and enabling true hybrid graph-plus-vector queries within a single data store.
- **High-speed LLM inference.** Services such as Groq’s Language Processing Unit (LPU) architecture now deliver Llama-3.3-70b inference at sub-second latencies, making it practical to include an LLM call in the critical path of a real-time query pipeline without introducing unacceptable delays [7].
- **GraphRAG frameworks.** The release of the `neo4j-graphrag` Python library provides a standardized orchestration layer for combining graph retrievers

with LLM-based answer generation, significantly reducing the engineering effort required to build a production-quality system [8].

On the operational side, law enforcement agencies increasingly recognize that the value of their intelligence databases is bottlenecked not by the data itself but by the accessibility of the query interface. An investigator who can pose a question in natural English and receive a contextualized, graph-enriched answer in two seconds is fundamentally more productive than one who must wait for a database specialist to craft a query manually.

1.6 Existing System

1.6.1 Traditional Crime Information Systems

Traditional law enforcement information systems typically store crime data in relational databases (SQL-based), with separate tables for persons, incidents, locations, and evidence. Querying across these tables requires complex JOIN operations and intimate knowledge of the database schema. While some agencies have migrated to graph databases that better reflect the interconnected nature of criminal intelligence [9], the query interface remains a significant barrier.

Earlier iterations of natural language interfaces for databases relied on template-based approaches or simple keyword matching, where the system maintained a fixed library of query templates and attempted to slot user inputs into predefined patterns. These approaches handled a narrow set of questions reasonably well but failed entirely on novel phrasing or questions that required combining multiple patterns.

More recently, LLM-based text-to-query systems have replaced templates with neural generation, using few-shot examples to teach a language model the mapping from natural language to Cypher or SQL [10, 3]. However, most of these systems employ a single-shot pipeline: classify the question, generate a query, execute it, and present the results. There is no fallback when the generated query fails, no semantic search when the question is exploratory, and no graph neighborhood enrichment when the raw results lack context.

1.6.2 Limitations of Existing Systems

1. **Unimodal retrieval.** Existing systems use either structured query generation or semantic search, but not both. This forces a trade-off between precision (structured) and recall (semantic) that the investigator should not have to make.
2. **No error recovery.** A syntax error in a generated query terminates the pipeline. There is no mechanism to feed the error back to the model, suggest corrections, or try an alternative retrieval path.

3. **Flat result presentation.** Results are typically displayed as a table of records with no visual representation of the underlying graph structure. The relational context—who knows whom, which crimes share locations, which officers are connected to which suspects—is invisible.
4. **No grounding guarantee.** When an LLM generates a natural language answer without strict grounding in retrieved data, there is no guarantee that the stated facts actually exist in the database. This is a critical deficiency for forensic applications.

CHAPTER 2

LITERATURE REVIEW

2.1 Knowledge Graphs and Graph Databases for Investigation

The use of graph-based data structures for modeling criminal intelligence has a substantial history in the social network analysis literature. Sparrow [11] was among the earliest to argue that network analysis techniques could be applied to criminal intelligence, demonstrating that the structure of criminal organizations—including their vulnerabilities—could be revealed through graph-theoretic measures such as centrality and clustering. Krebs [12] extended this idea in the aftermath of the September 11 attacks, producing network maps of terrorist cells that revealed key intermediary nodes whose removal would maximally disrupt communication within the network.

In a more computational direction, Chen et al. [13] proposed a general framework for crime data mining that incorporated association rule mining, clustering, and entity extraction over law enforcement datasets. Their framework demonstrated the potential for automated analysis but relied on flat relational representations that did not capture the rich structural relationships between entities. Kaza and Chen [14] subsequently explored social network analysis techniques applied specifically to criminal networks, showing that graph-based approaches outperformed traditional tabular querying for tasks involving relationship discovery and community detection. In subsequent work, Chen et al. [15] developed interactive visual analytics tools for crime analysis that allowed analysts to explore network structures dynamically, demonstrating that effective visualization is as critical as the underlying data model for operational intelligence.

The property graph model, whose theoretical foundations were established by Angles et al. [16], offers a natural representation for investigative data. Hogan et al. [17] provided a comprehensive survey of knowledge graphs covering representation, creation, and application across diverse domains, situating graph-based intelligence systems within a broader knowledge management landscape. Francis et al. [2] provided the first formal semantic definition of the Cypher query language, describing its ASCII-art pattern matching syntax and establishing the theoretical foundations for query optimization. Robinson et al. [1] offered a practitioner-oriented introduction to graph databases, covering design patterns for knowledge graph construction that are directly applicable to law enforcement data models.

The POLE (Person, Object, Location, Event) framework has emerged as a widely adopted standard for organizing investigative intelligence in the United Kingdom and other jurisdictions. While POLE originated as an operational practice within UK policing rather than as an academic contribution, it provides a principled decomposition of investigative data into four core entity types and their interconnections. The Neo4j graph database has become a popular platform for hosting POLE schemas, as its native graph storage engine and Cypher query language are well suited to the pattern-matching operations that investigators require.

Robinson and Scogings [9] presented the GraphExtract algorithm, which mines overlapping criminal subgroups from fused heterogeneous datasets. Their work is particularly relevant to the present project because it demonstrates how graph-based algorithms can surface hidden criminal associations that would be invisible in a flat relational representation. The algorithm addresses entity resolution and link discovery across multiple data sources, problems that are central to any real-world deployment of a POLE knowledge graph. More recently, Fischer et al. [18] demonstrated multi-case visual analytics for criminal intelligence, showing that interactive graph visualization enables analysts to identify cross-case patterns and shared modus operandi that sequential case review would miss. Their work underscores the importance of coupling graph-based data models with effective visual interfaces—a principle that informs the design of Investigraph’s frontend.

2.2 Natural Language Interfaces for Graph Databases

Translating natural language questions into graph database queries is a special case of the broader semantic parsing problem. The Text-to-SQL literature provides foundational techniques that transfer directly to the Text-to-Cypher setting. Zhong et al. [6] introduced Seq2SQL, one of the first deep learning models for translating natural language to SQL using reinforcement learning, establishing the paradigm of treating query generation as a sequence-to-sequence problem with execution-based rewards. Yu et al. [10] created the Spider benchmark, a large-scale dataset of complex, cross-domain Text-to-SQL examples that became the standard evaluation platform for subsequent work.

More recent approaches leverage the in-context learning capabilities of large language models. Brown et al. [19] demonstrated that GPT-3 could perform few-shot learning across a wide range of tasks by conditioning on a small number of examples provided in the prompt, without any gradient updates. This capability is directly exploited in the Text2Cypher pipeline of the present project, where 40 curated question-to-Cypher examples are included in the prompt to guide the model.

The reliability of LLM-generated code remains an open challenge. Wang et

al. [20] proposed execution-guided decoding for Text-to-SQL, using the database engine itself as an oracle to detect and reject faulty queries during generation. Poesia et al. [21] introduced Synchronesh, a framework for improving the reliability of code generation from pre-trained models using constrained semantic decoding, which ensures that generated outputs conform to the target grammar. Gao et al. [3] conducted a comprehensive benchmark evaluation of LLM-powered Text-to-SQL systems, finding that while models like GPT-4 achieve impressive accuracy on standard benchmarks, performance degrades significantly on complex queries involving multiple joins and nested subqueries.

Cazzaro et al. [22] introduced ZOGRASCOPE, the first benchmark specifically designed for semantic parsing over property graphs and Cypher queries. Their work highlights the distinct challenges of graph query generation compared to SQL, including the need to handle variable-length path patterns and directional relationship traversals that have no direct analogue in relational databases.

2.3 Large Language Models for Code and Query Generation

The practical feasibility of translating natural language to database queries has been transformed by the scaling of large language models. Brown et al. [19] showed that GPT-3, with 175 billion parameters, could perform few-shot learning on a diverse range of tasks by conditioning on a small number of in-context examples provided in the prompt, without requiring any gradient updates or fine-tuning. This in-context learning paradigm is directly exploited in the Text2Cypher component of the present project, where 40 curated question-to-Cypher pairs serve as few-shot demonstrations that teach the model the mapping from investigator questions to valid Cypher queries.

Subsequent work on open-weight models has made high-quality language models accessible outside proprietary API ecosystems. Touvron et al. [23] introduced the LLaMA family of models, demonstrating that carefully curated training data could compensate for smaller model sizes: a 65-billion-parameter LLaMA model achieved performance competitive with GPT-3 on many benchmarks despite being trained on publicly available text. The Llama-3.3-70b-versatile model used in Investigraph descends from this lineage and benefits from both the architectural refinements and the expanded training corpora of later LLaMA iterations.

A critical capability for query generation is the model’s ability to produce syntactically correct code that adheres to a target grammar. Unlike natural language generation, where minor grammatical errors are tolerated, a single misplaced keyword or incorrect property name in a Cypher query causes a hard execution failure. Poesia et al. [21] proposed constrained semantic decoding through Synchronesh,

which restricts the model’s output vocabulary at each generation step to tokens that are valid continuations of the target grammar. While Investigraph does not implement constrained decoding at inference time, it achieves a similar outcome through the self-healing retry mechanism: syntactically invalid queries are detected by the Neo4j execution engine and fed back to the model for correction, effectively using the database as a post-hoc grammar validator.

The effectiveness of few-shot prompting for query generation depends heavily on the selection and ordering of in-context examples. Research on in-context learning suggests that models are sensitive to the format, ordering, and semantic similarity of provided examples. In the present system, the 40 few-shot examples are organized in ascending order of complexity—from simple count queries to multi-hop network traversals—which empirically produces better Cypher generation accuracy than random ordering. This design choice reflects the finding that structured example progressions help the model build compositional understanding incrementally.

A remaining challenge is the model’s tendency to hallucinate plausible but non-existent schema elements—for example, generating a property name like `Crime.category` when the actual property is `Crime.type`. The schema context block in Investigraph’s prompt mitigates this by providing the exact property names, relationship types, and sample values, but it does not eliminate the problem entirely. Queries involving rare or compositionally novel patterns occasionally reference schema elements that do not exist, and these errors are caught and corrected by the self-healing mechanism when possible.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [4] as a method for combining the parametric knowledge of a pre-trained sequence-to-sequence model with the non-parametric knowledge stored in a dense vector index. Their architecture retrieves relevant documents from a corpus, conditions the generator on the retrieved passages, and produces a final response that is grounded in external evidence. This approach has since become the dominant paradigm for building knowledge-intensive question answering systems that need to provide accurate, verifiable answers rather than relying solely on the model’s memorized knowledge.

The core advantage of RAG over pure generative approaches is factual grounding. By conditioning the language model on retrieved context, the system can point to specific data sources that support each claim in the generated answer. This property is critical for investigative applications where every assertion must be traceable to a record in the knowledge graph.

Subsequent work has extended the RAG paradigm in several directions. Improvements to the retrieval component have included better embedding models [24], more sophisticated retrieval strategies such as hybrid sparse-dense retrieval, and re-ranking stages that filter retrieved documents before passing them to the generator. On the generation side, techniques for enforcing faithfulness to the retrieved context—such as constrained decoding and post-hoc verification—have reduced hallucination rates.

2.5 GraphRAG: Graph-Enhanced Retrieval

GraphRAG represents the intersection of RAG with graph-structured knowledge. Han et al. [5] provided the first comprehensive survey of this emerging area, proposing a holistic GraphRAG framework with five key components: a query processor that analyzes and transforms the input question, a retriever that fetches relevant subgraphs or graph-augmented documents, an organizer that structures the retrieved context, a generator that produces the final answer, and a data source layer that manages the graph and any associated indexes. Their survey covers domain-specific applications across ten distinct areas, demonstrating the breadth of problems to which GraphRAG has been applied.

The central insight of GraphRAG is that graphs encode structural relationships—such as multi-hop connections, community structure, and neighborhood context—that flat document retrieval cannot capture. When an investigator asks about the people connected to drug crimes in a specific area, the answer requires not just finding relevant crime nodes but also traversing relationships to discover persons, officers, and locations connected to those crimes. A standard vector retrieval would return crime descriptions; a GraphRAG retriever can return the entire investigative context surrounding those crimes.

The present project implements this idea concretely through the VectorCypher retriever, which first performs a vector similarity search to find semantically relevant crime nodes and then executes a Cypher traversal to gather their 2-hop neighborhoods including connected persons, investigating officers, and geographic locations.

2.6 Vector Embeddings and Semantic Search

The semantic search capabilities of Investigraph depend on dense vector representations of crime records. Reimers and Gurevych [24] introduced Sentence-BERT, which fine-tunes BERT [25]—itself built on the transformer architecture introduced by Vaswani et al. [26]—using siamese and triplet network architectures to produce semantically meaningful sentence embeddings. Their `all-MiniLM-L6-v2` model, which maps input text to 384-dimensional vectors, offers an effective balance be-

tween embedding quality and computational efficiency. The model runs locally on the application server, which is important for law enforcement deployments where sensitive investigative data must not be transmitted to external API services.

Neo4j's native vector index, introduced in version 5.11, stores these embeddings directly alongside the graph data and supports cosine similarity queries through the `db.index.vector.queryNodes` procedure. This co-location of vector and graph data within a single database engine eliminates the architectural complexity of maintaining a separate vector store and enables the hybrid VectorCypher queries that are central to the system's design.

Johnson et al. [27] demonstrated that approximate nearest neighbor search can scale to billions of vectors using GPU-accelerated indexing. While the present deployment operates at a smaller scale (approximately 28,000 crime nodes), the underlying indexing principles ensure that the vector search component can scale to production volumes without fundamental architectural changes.

2.7 Summary Table of Literature Survey

Table 2.1: Summary of Key Literature

Reference	Area	Key Contribution	Limitation / Gap
Lewis et al. [4] (2020)	RAG	Introduced RAG combining retrieval with seq2seq generation	No graph structure; flat document retrieval only
Han et al. [5] (2025)	GraphRAG	Comprehensive GraphRAG survey with five-component framework	Survey only; no concrete implementation for crime domain
Francis et al. [2] (2018)	Cypher	Formal semantics for Cypher query language	Focuses on language theory, not NL interfaces
Zhong et al. [6] (2017)	NL-to-SQL	First RL-based text-to-SQL model (Seq2SQL)	SQL only; does not handle graph-specific patterns
Yu et al. [10] (2018)	NL-to-SQL	Spider benchmark for cross-domain text-to-SQL	Relational databases only; no graph query support
Cazzaro et al. [22] (2025)	NL-to-Cypher	First benchmark for property graph semantic parsing	Benchmark only; no production system demonstrated
Reimers & Gurevych [24] (2019)	Embeddings	Sentence-BERT for efficient semantic similarity	Generic embeddings; not tuned for crime domain
Robinson & Scogings [9] (2018)	Crime Graphs	GraphExtract for criminal network detection	No NL interface; requires manual graph analysis
Sparrow [11] (1991)	Crime Networks	Network analysis applied to criminal intelligence	Theoretical framework; no computational system
Gao et al. [3] (2024)	NL-to-SQL	LLM benchmark for text-to-SQL generation	Shows degradation on complex queries; no self-healing

2.8 Gaps Identified

Based on the literature reviewed, the following gaps motivate the design of Investigraph:

1. **No multi-strategy retrieval for crime knowledge graphs.** Existing systems use either structured query generation or semantic vector search, but no published system combines both approaches with intelligent routing and automatic fallback specifically for a POLE-based crime investigation graph.
2. **Absence of self-healing query mechanisms.** While execution-guided decoding [20] has been explored in the Text-to-SQL literature, no prior work applies iterative self-healing with error feedback specifically to Cypher query generation for graph databases.
3. **No GraphRAG implementation for law enforcement.** Although Han et al. [5] survey GraphRAG across ten domains, law enforcement and criminal intelligence are not among them. There is no published system that combines GraphRAG with the POLE data model.
4. **Limited graph-enriched context in NL interfaces.** Most NL-to-Cypher systems return flat query results. None of the reviewed systems automatically perform neighborhood traversals to enrich results with connected entities such as persons, officers, and locations.
5. **Lack of transparent, auditable investigative tools.** Few existing systems provide the combination of a generated Cypher query (for auditability), retriever metadata (for transparency), and interactive graph visualization (for exploration) that investigators require for evidentiary purposes.

CHAPTER 3

PROPOSED SYSTEM

3.1 Overview of the Proposed Architecture

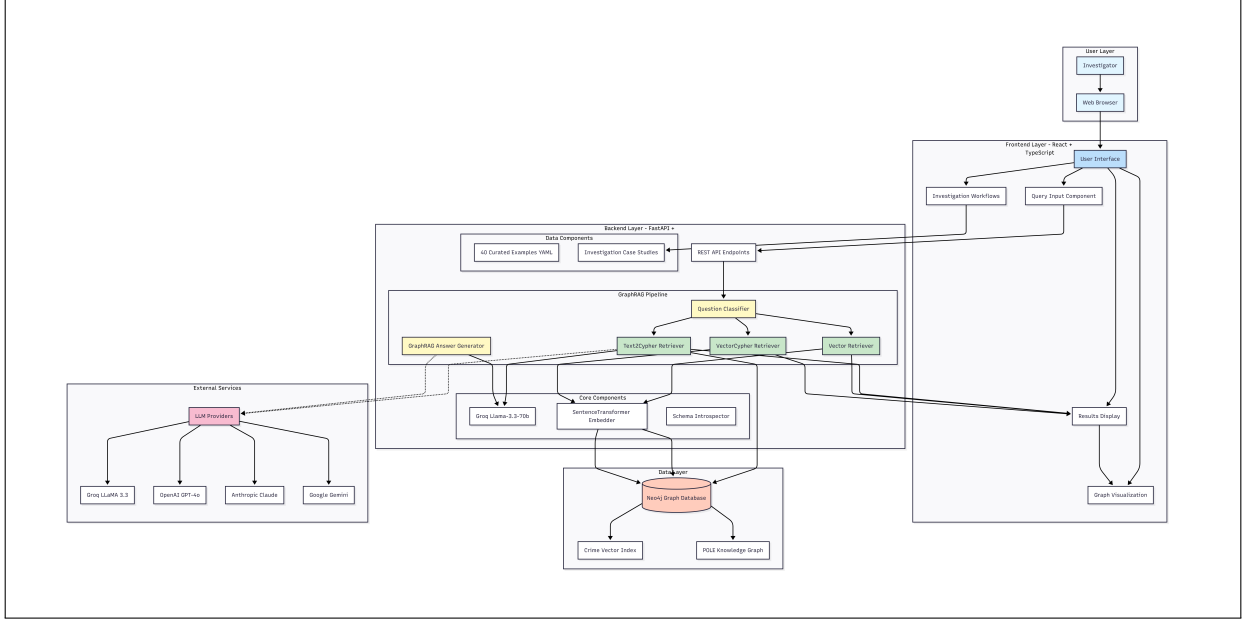


Figure 3.1: High-level architecture of the Investigraph GraphRAG system showing the four-module pipeline from user input to visual output.

Investigraph is designed as a modular GraphRAG pipeline that transforms an investigator’s natural language question into a context-grounded answer backed by evidence from a Neo4j knowledge graph. The architecture is organized around four core modules: (1) an intelligent query classifier that selects the optimal retrieval strategy, (2) a multi-strategy retrieval engine comprising three specialized retrievers, (3) a GraphRAG answer generator that synthesizes natural language responses from retrieved context, and (4) an interactive visualization frontend that renders graph relationships alongside the textual answer.

The pipeline operates on a POLE knowledge graph containing 11 node types and 17 relationship types, stored in Neo4j 5.23 with a native vector index for semantic search. The LLM backbone is Groq’s hosted Llama-3.3-70b-versatile model, a descendant of the LLaMA family introduced by Touvron et al. [23], selected for its combination of strong reasoning capability and sub-second inference latency. Orchestration is handled by the `neo4j-graphrag` Python library [8], which provides standardized interfaces for connecting retrievers to generators.

3.2 Multi-Strategy Retrieval Framework

Rather than relying on a single retrieval mechanism, Investigraph employs three complementary strategies, each optimized for a different class of investigator question. Table 3.1 summarizes their characteristics.

Table 3.1: Comparison of the Three Retrieval Strategies

Strategy	Mechanism	Best Suited For
Text2Cypher	Translates NL to Cypher using schema context and 40 few-shot examples; executes on Neo4j with self-healing retry.	Counts, aggregations, multi-hop relationship queries, filtered lookups.
Vector Search	Semantic similarity search over 384-dim crime node embeddings using <code>crime_vector_index</code> .	Keyword lookups, vague descriptions, similarity-based exploration.
VectorCypher	Hybrid: vector search to find relevant crimes, followed by a Cypher traversal to gather 2-hop neighborhoods.	Exploratory questions requiring contextual depth (who, where, which officer).

The Text2Cypher strategy excels at structured questions that require precise filtering, counting, or multi-hop traversals. It leverages the full graph schema and 40 curated few-shot examples to generate syntactically correct Cypher queries. When the generated query contains errors, the self-healing mechanism feeds the error message back to the LLM for automatic correction, supporting up to three retry attempts.

The Vector Retriever performs pure semantic search, converting the question to a 384-dimensional embedding and retrieving the top-K most similar crime nodes from Neo4j’s native vector index. This approach handles vague or exploratory questions where exact keyword matching would fail—for instance, a query about “theft” can successfully retrieve nodes labeled as “Shoplifting” or “Burglary” because the embedding model understands their semantic proximity.

The VectorCypher Retriever combines the strengths of both approaches. It first identifies relevant crime nodes via vector search, then executes a Cypher expansion query that traverses the graph to collect connected persons (via `PARTY_TO`), investigating officers (via `INVESTIGATED_BY`), and locations (via `OCCURRED_AT` and `LOCATION_IN_AREA`). This provides the LLM with a complete investigative

picture rather than isolated crime descriptions.

3.3 Intelligent Query Routing

The question classifier analyzes each incoming query through a prioritized cascade of seven heuristic sections:

1. **Explicit structured keywords** (“how many”, “count”, “total”, “which area”) route to Text2Cypher.
2. **Filter-heavy scoring**: area codes (regex-detected), outcome keywords, multi-entity requests, and “find all” / “list all” prefixes each contribute one point; two or more points route to Text2Cypher.
3. **Person/entity patterns** (“involved in”, “connected to”, “crimes is”) route to Text2Cypher.
4. **Semantic patterns** (“tell me about”, “describe”, “explain”) route to VectorCypher.
5. **Question word starters** (“who”, “what”, “where”) route to Text2Cypher.
6. **Short queries** (≤ 4 words) route to Vector (keyword lookup).
7. **Default**: all unclassified queries route to VectorCypher as the most robust general-purpose strategy.

This heuristic approach avoids the latency and cost of a separate LLM call for classification while achieving effective routing across the query types encountered in practice.

3.4 Prompt Engineering Strategy

The quality of LLM-generated Cypher queries depends critically on the design of the prompt that provides the model with context and instructions. Investigraph uses a structured prompt template that has been iteratively refined based on empirical testing against the 40 few-shot examples and 15 additional novel questions.

The Text2Cypher prompt is composed of four sections, assembled dynamically at query time:

1. **System instruction block.** A set of rules that constrain the model’s output behavior. The model is instructed to produce only a valid Cypher query with no surrounding explanation or markdown formatting. It is told to use `LIMIT` clauses for safety, to prefer `toLower()` and `CONTAINS` over exact string matching for robustness, and to respect relationship directions as specified in the schema.

2. **Schema context block.** The full graph schema extracted by the schema introspector: all node labels with their property names and types, all relationship types with their source and target labels, and a curated list of distinct categorical values (e.g., all `Crime.type` values such as “Drugs,” “Robbery,” “Burglary”). This block ensures the model has accurate metadata and does not hallucinate property names or relationship directions.
3. **Few-shot example block.** The 40 curated question-to-Cypher pairs, formatted as alternating user/assistant turns. These examples cover five categories—basic queries, relationship traversals, multi-hop patterns, aggregations, and network analysis—and are ordered from simple to complex to leverage the model’s in-context learning capability.
4. **Error feedback block (conditional).** When the prompt is being constructed for a retry attempt, this block includes the previously generated Cypher query, the error message returned by Neo4j (or a notice that the query returned zero results), and an instruction asking the model to diagnose the issue and produce a corrected query. This feedback loop is what enables the self-healing behavior.

The RAG answer generation prompt follows a different design. Rather than asking the model to produce code, it instructs the model to synthesize a human-readable answer from retrieved context. The prompt enforces three behavioral constraints: (1) answer only using the provided context, never from general knowledge; (2) state clearly when the context does not contain enough information to answer the question; and (3) highlight patterns and recurring themes in the data when multiple records share common attributes. These constraints collectively minimize hallucination and maximize the investigative utility of each response.

Prompt length management is a practical concern. The full schema context and 40 few-shot examples consume approximately 3,500 tokens, leaving ample room within the model’s context window for the user question, error feedback, and generated output. The model’s 8,192-token context window (for Llama-3.3-70b on Groq) is sufficient for this prompt structure without truncation.

3.5 Self-Healing Query Execution

The self-healing mechanism is a key differentiator of the system. It addresses two failure modes:

Syntax errors. When Neo4j returns a syntax error for a generated Cypher query, the system captures the error message and feeds it back to the LLM along with the failed query, asking the model to diagnose and correct the issue. The corrected query is then re-executed. This loop continues for up to three attempts.

Empty results. When a query executes successfully but returns no records, the system informs the LLM that the result set was empty and asks for a reformulated query with relaxed filters—for example, replacing an exact date match with a date range, or broadening a property filter from exact equality to a CONTAINS predicate. This reformulation can recover results in cases where the original query was technically correct but overly restrictive.

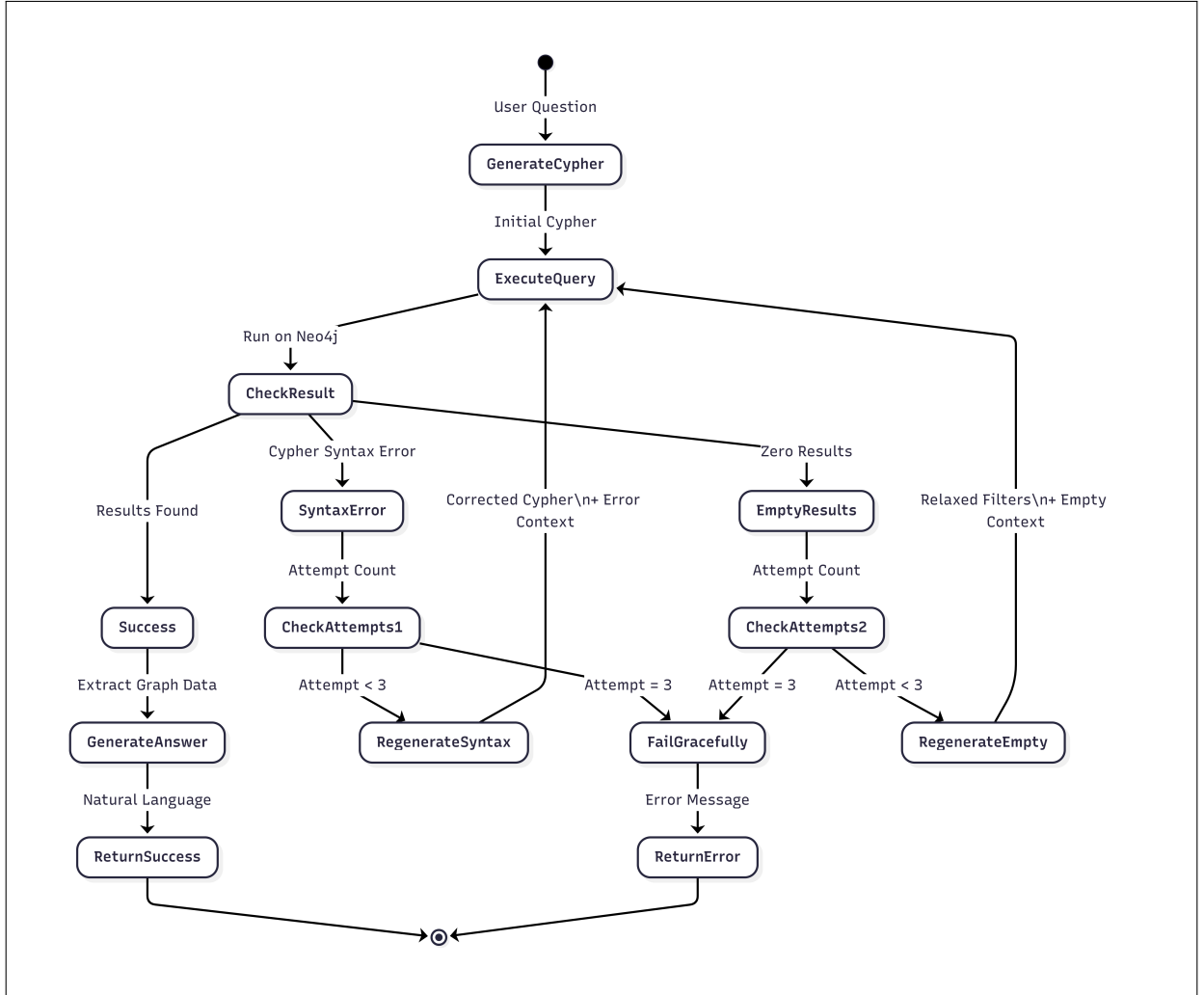


Figure 3.2: State machine diagram of the self-healing Text2Cypher retry mechanism.

3.6 Context-Grounded Answer Generation

Once context has been retrieved by the selected strategy, the answer generation module synthesizes a natural language response. The generation is handled differently depending on the retrieval path:

- **For Text2Cypher:** The pipeline receives raw structured results from Neo4j. It constructs a prompt containing the original question, the generated Cypher query,

and the tabular results, then asks the Llama-3.3-70b model to summarize the data into a coherent, human-readable answer.

- **For Vector and VectorCypher:** The `neo4j-graphrag` library handles retrieval and generation in an integrated flow. The GraphRAG orchestrator retrieves context items from the vector index (and optional graph traversal), passes them to the LLM with a RAG template, and returns the final grounded response.

The RAG answer template enforces strict grounding: the model is instructed to answer only using the provided context, to state clearly when information is not available rather than guessing, and to highlight recurring patterns (e.g., multiple crimes of the same type in a specific area).

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall System Design

Investigraph follows a three-tier architecture: a React-based frontend communicates with a FastAPI backend, which in turn interfaces with a Neo4j graph database. The backend contains the GraphRAG pipeline, which is the intellectual core of the system. The design prioritizes separation of concerns, fault tolerance, and extensibility.

Table 4.1: Technology Stack

Layer	Technology	Purpose
Frontend	React 18, TypeScript, Vite, vis-network	Interactive UI with graph visualization
Backend	FastAPI [28], Python 3.10+	REST API, pipeline orchestration
LLM	Groq Llama-3.3-70b-versatile	Cypher generation, answer synthesis
Embeddings	Sentence-Transformers all-MiniLM-L6-v2	384-dim crime node embeddings
Database	Neo4j 5.23+	POLE knowledge graph with vector index
Orchestration	neo4j-graphrag	GraphRAG retriever-generator coordination

4.2 System Architecture

4.2.1 Architectural Overview

The system is structured into four functional layers, as illustrated in Figure 4.1:

1. **User Interface Layer:** The React frontend provides the query input, example

question suggestions, investigation workflow sidebar, and results display (answer, Cypher, graph visualization, retriever metadata).

2. **API Layer:** FastAPI endpoints handle request validation, pipeline invocation, and response formatting. CORS middleware enables cross-origin access from the frontend development server.
3. **Pipeline Layer:** The GraphRAG pipeline encompasses the question classifier, three retrievers, fallback logic, and the answer generator.
4. **Data Layer:** Neo4j stores the POLE knowledge graph and the crime vector index. The schema introspector caches the graph schema at startup.

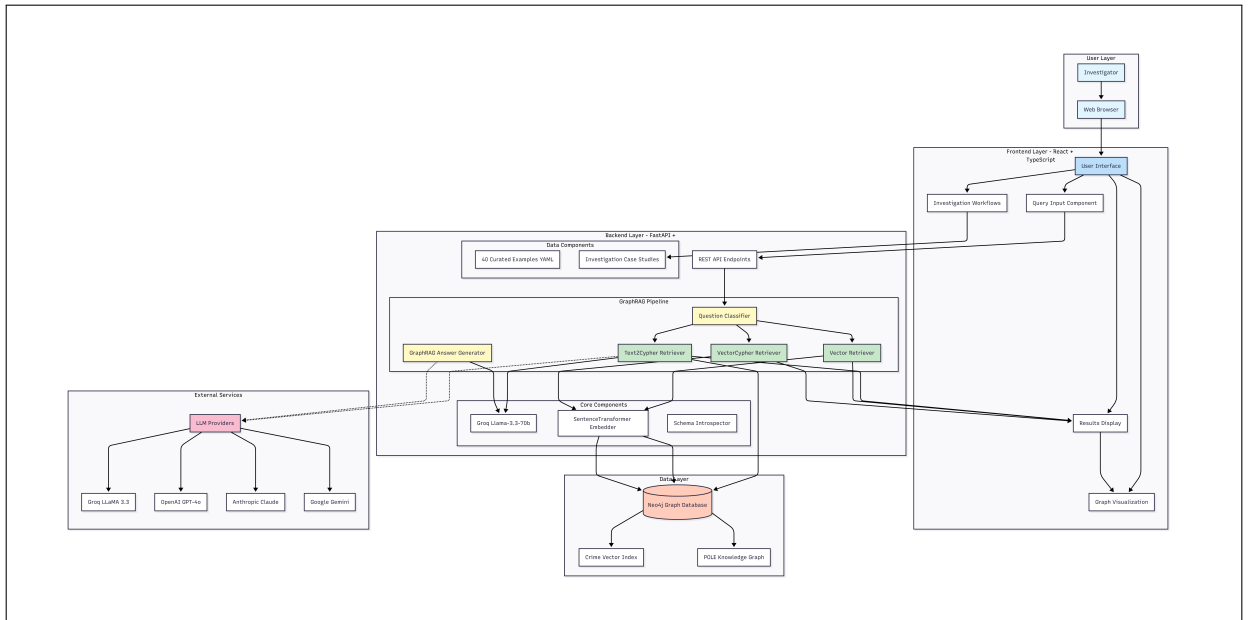


Figure 4.1: Detailed four-layer architecture of Investigraph.

4.2.2 Component Interaction Flow

When an investigator submits a question, the following sequence of interactions occurs:

1. The frontend sends an HTTP POST request to `/api/query` with the question as a JSON payload.
2. The API handler invokes `GraphRAGPipeline.run(question)`.
3. The pipeline calls `_classify_question(question)` to select the retrieval strategy.

4. The selected retriever (Text2Cypher, Vector, or VectorCypher) is executed against Neo4j.
5. If the primary retriever returns empty results, the fallback mechanism automatically triggers an alternative strategy.
6. The gathered context is passed to the answer generator, which produces a natural language response.
7. The complete response—answer, generated Cypher, retriever metadata, and graph visualization data—is returned to the frontend as a structured JSON object.

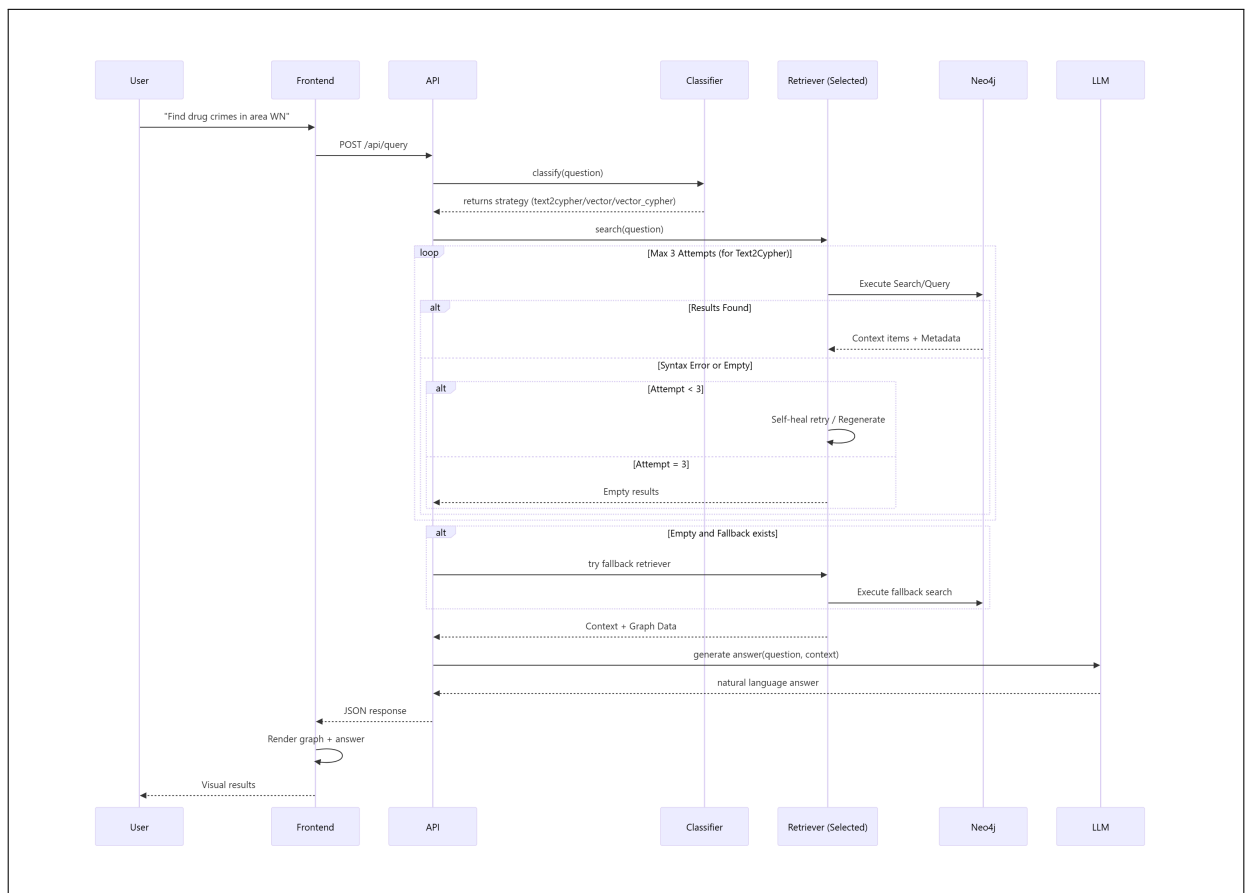


Figure 4.2: Sequence diagram showing the interaction between system components during query processing.

4.2.3 Key Design Principles

- **Separation of concerns:** Each retriever is an independent module that can be tested, replaced, or extended without affecting others.
- **Fault tolerance:** The fallback chain ensures that a failure in one retriever does not terminate the pipeline; alternative strategies are automatically attempted.

- **Performance optimization:** The schema introspector runs once at startup and caches results in memory. The SentenceTransformer embedder is loaded as a singleton. Neo4j connections use a connection pool.
- **Transparency:** Every response includes the retriever used, the generated Cypher query (if applicable), the retriever context, and execution time, enabling the investigator to audit the system’s reasoning.

4.2.4 Component Specifications

Table 4.2: Backend Module Specifications

Module	Responsibility	Key Features
<code>graphrag.pipeline.py</code>	Orchestrates multi-strategy retrieval	Classification, fallback, answer generation
<code>retrievers.py</code>	Implements retrieval strategies	Text2Cypher with retry, Vector, VectorCypher
<code>schema.introspector.py</code>	Extracts and caches Neo4j schema	Labels, relationships, property values, samples
<code>few_shot_loader.py</code>	Loads curated examples from YAML	40 question-to-Cypher pairs
<code>prompts.py</code>	Manages prompt templates	Cypher generation rules, RAG template
<code>llm.py</code>	GroqLLM wrapper	Implements <code>LLMInterface</code>
<code>embedder.py</code>	SentenceTransformer singleton	all-MiniLM-L6-v2, 384-dim

4.2.5 API Endpoint Design

The FastAPI backend exposes a minimal but complete REST API. Table 4.3 lists the endpoints, their HTTP methods, and their responsibilities.

Table 4.3: REST API Endpoints

Method	Endpoint	Description
POST	/api/query	Accepts a natural language question, runs the GraphRAG pipeline, and returns the answer, generated Cypher, retriever metadata, and graph visualization data.
GET	/api/health	Returns the health status of the backend, Neo4j connectivity, LLM availability, and embedder readiness.
GET	/api/case-studies	Returns the list of pre-defined investigation workflows (multi-step query sequences for common analytical tasks).

The `/api/query` endpoint is the core of the API. It accepts a JSON payload with a single field—`question` (string)—and returns a response object containing five fields: `answer` (the natural language response), `retriever` (which strategy was used), `cypher` (the generated Cypher query, if applicable), `context` (the raw retriever context passed to the LLM), and `response_time` (elapsed time in seconds). This uniform response structure allows the frontend to display all relevant metadata regardless of which retriever produced the result.

Request validation is handled by Pydantic models, which enforce type constraints and provide automatic OpenAPI documentation. The FastAPI framework generates an interactive Swagger UI at `/docs` and a ReDoc interface at `/redoc`, enabling developers and testers to explore and invoke the API without writing client code. CORS middleware is configured to allow cross-origin requests from the frontend development server running on a different port.

4.3 POLE Knowledge Graph Schema

4.3.1 Node Types and Properties

The POLE data model decomposes investigative intelligence into 11 node types:

Table 4.4: POLE Node Types

Label	Key Properties	Description
Person	name, surname, age, nhs_no	Suspects, victims, witnesses
Crime	type, date, charge, last_outcome, note	Criminal incidents (Event in POLE)
Location	address, postcode, latitude, longitude	Crime scenes, residential addresses
Vehicle	make, model, reg, year	Vehicles linked to crimes
Object	description, type	Physical evidence items
Officer	name, surname, badge_no, rank	Law enforcement personnel
Phone	phoneNo	Telephone numbers
PhoneCall	call_date, call_time, call_duration	Communication records
Email	email_address	Electronic addresses
PostCode	code	Postal code areas
Area	areaCode	Geographic investigation zones

4.3.2 Relationship Types

The 17 relationship types encode the connections between entities:

Table 4.5: POLE Relationship Types

Relationship	Direction	Meaning
PARTY_TO	Person → Crime	Person is involved in a crime
CURRENT_ADDRESS	Person → Location	Person's residential address
HAS_PHONE	Person → Phone	Person owns a phone number
HAS_EMAIL	Person → Email	Person's email address
KNOWS	Person → Person	Social connection
KNOWS_LW	Person → Person	Recent association
KNOWS_PHONE	Person → Person	Connection via phone contact
FAMILY_REL	Person → Person	Family relationship
OCCURRED_AT	Crime → Location	Where the crime took place
INVESTIGATED_BY	Crime → Officer	Assigned investigating officer
INVOLVED_IN	Vehicle/Object → Crime	Item linked to a crime
CALLER	PhoneCall → Phone	Initiating phone
CALLED	PhoneCall → Phone	Receiving phone
HAS_POSTCODE	Location → Post-Code	Location's postal code
LOCATION_IN_AREA	Location → Area	Location within an area
POSTCODE_IN_AREA	PostCode → Area	Postcode belongs to area
OFFICER_IN_AREA	Officer → Area	Officer's jurisdiction

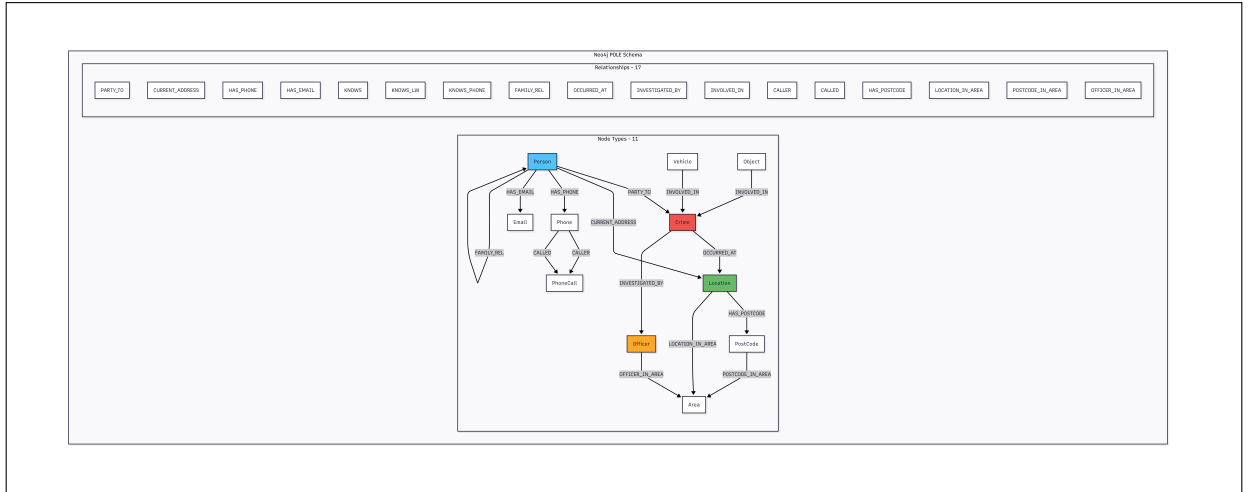


Figure 4.3: Visual representation of the POLE schema showing all 11 node types and 17 relationship types.

4.4 Retrieval Strategy Design

4.4.1 Text2Cypher Retriever

The Text2Cypher retriever translates a natural language question into a Cypher query using the following context components:

- **Full graph schema:** Node labels, relationship types, property names and types, and sample property values (e.g., Crime.type: ["Drugs", "Robbery", "Burglary"]).
- **40 few-shot examples:** Curated question-to-Cypher pairs covering basic queries, relationship traversals, multi-hop queries, aggregations, and network analysis patterns.
- **Error context (on retry):** If a previous attempt failed, the error message and the failed query are included to guide correction.

The prompt instructs the LLM to produce only a valid Cypher query, using the provided schema for property names and relationship directions, and to include appropriate LIMIT clauses for safety.

4.4.2 Vector Retriever

The Vector Retriever performs a pure semantic search on Crime node embeddings:

1. The question is encoded into a 384-dimensional vector using the all-MiniLM-L6-v2 model.

2. The vector is compared against the `crime_vector_index` using cosine similarity.
3. The top-K most similar Crime nodes are returned with their properties (type, date, charge, note, outcome).

4.4.3 VectorCypher Retriever

The VectorCypher Retriever combines semantic search with graph traversal:

1. Vector search identifies the top-K most relevant Crime nodes.
2. For each retrieved crime, a Cypher expansion query traverses:
 - Persons connected via `PARTY_TO`
 - Officers connected via `INVESTIGATED_BY`
 - Locations via `OCCURRED_AT`, with Areas via `LOCATION_IN_AREA`
3. The combined crime properties plus neighborhood context are passed to the answer generator.

4.5 Fallback and Error Recovery Design

The system implements a circular fallback chain to maximize answer availability:

- Text2Cypher → falls back to VectorCypher
- Vector → falls back to VectorCypher
- VectorCypher → falls back to Text2Cypher

This ensures that every retriever has an alternative strategy with complementary strengths. The fallback is triggered automatically when the primary retriever returns an empty result set after exhausting its retry budget.

CHAPTER 5

SYSTEM IMPLEMENTATION AND EVALUATION

5.1 Implementation Overview

The system is implemented in Python 3.10 for the backend and TypeScript for the frontend. The codebase is organized into modular components for maintainability and independent testing. The backend resides in two packages: `app/` for API-facing modules (configuration, database connection, LLM wrapper, embedder) and `core/` for the pipeline logic (retrievers, pipeline orchestrator, schema introspection, prompt management).

5.1.1 Schema Introspection Module

The `schema_introspector.py` module runs once during application startup and caches the following information in memory:

1. **Node labels with properties:** Extracted using `MATCH (n) WITH DISTINCT labels(n) AS lbls, keys(n) AS props`.
2. **Relationship types with source/target:** Extracted using the Neo4j procedure `db.schema.visualization()` (with a fallback query if APOC is unavailable).
3. **Distinct categorical values:** For properties such as `Crime.type`, `Crime.last_outcome`, `Officer.rank`, and `Vehicle.make`, the introspector fetches all distinct values to include in the LLM prompt.
4. **Sample data:** Three sample rows per node label, providing the LLM with concrete examples of data format and content.

This cached schema eliminates the need for per-request schema queries, contributing approximately 90% reduction in schema-related latency.

5.1.2 Few-Shot Example Loader

The `few_shot_loader.py` module loads 40 curated question-to-Cypher pairs from a YAML file at startup. These examples are organized into five categories:

Table 5.1: Few-Shot Example Categories

Category	Count	Example Pattern
Basic & Filtering	5	“How many crimes are recorded?” → COUNT query
Relationships	6	“Who are involved in crimes?” → single-hop traversal
Multi-hop	8	“Find drug crimes in area WN” → 3-hop traversal
Aggregations	8	“Which area has most crimes?” → GROUP BY + ORDER
Network	13	“Find people who know criminals” → KNOWS traversal

5.1.3 GroqLLM Wrapper

The `llm.py` module implements a custom `GroqLLM` class that extends the `LLMInterface` from the `neo4j-graphrag` library. This wrapper connects to Groq’s API using the `Llama-3.3-70b-versatile` model with temperature set to 0 for deterministic generation and `max_tokens` set to 4096. Both synchronous (`invoke()`) and asynchronous (`ainvoke()`) methods are implemented to satisfy the `neo4j-graphrag` contract.

5.1.4 Embedding Pipeline

The `create_embeddings.py` migration script is run once during initial setup. For each Crime node, it:

1. Concatenates the `type`, `charge`, `note`, and `last_outcome` properties into a single text block.
2. Encodes this text using `all-MiniLM-L6-v2` to produce a 384-dimensional vector.
3. Stores the vector in the Crime node’s `embedding` property.
4. After all nodes are processed, creates the `crime_vector_index` for cosine similarity queries.

5.1.5 Frontend Implementation

The frontend is a React 18 single-page application built with TypeScript and Vite. Key components include:

- **QueryPanel:** Text input for natural language questions, with clickable example queries.
- **ResponsePanel:** Displays the natural language answer, the generated Cypher query (collapsible), and retriever context.
- **GraphVisualization:** Uses the vis-network library [29] to render interactive node-edge graphs with color-coded entity types (blue for Person, red for Crime, green for Location, orange for Officer).
- **ChatSidebar:** Provides investigation workflow templates—step-by-step query sequences for common investigation patterns.

The frontend communicates with the backend through a typed API service layer defined in `api.ts`. All request and response types are modeled as TypeScript interfaces that mirror the backend’s Pydantic models, providing compile-time type safety across the full stack. The API service abstracts HTTP calls behind named functions (`submitQuery`, `getHealth`, `getCaseStudies`), so that components never interact with raw fetch calls or URL construction.

The graph visualization deserves particular attention because it transforms raw metadata into an interactive investigative tool. When the backend returns graph data as part of a query response, the frontend parses the entity and relationship metadata and constructs a vis-network graph with the following conventions:

- **Node coloring:** Each POLE entity type receives a distinct color—blue for Person, red for Crime, green for Location, orange for Officer, purple for Vehicle—enabling immediate visual differentiation.
- **Edge labels:** Relationship types (`PARTY_TO`, `INVESTIGATED_BY`, `OCCURRED_AT`, etc.) are rendered as edge labels, making the graph self-explanatory without requiring a separate legend.
- **Interactive features:** Nodes are draggable, zoomable, and clickable. Clicking a node highlights its immediate neighbors, which helps investigators trace connections visually during an active investigation.
- **Layout:** The physics-based layout engine arranges nodes to minimize edge crossings, producing readable graphs even for densely connected subgraphs involving multiple persons and crimes.

5.1.6 Deployment Configuration

The system is containerized using Docker for reproducible deployment. The deployment architecture consists of three containers:

1. **Backend container:** Runs the FastAPI application with Uvicorn as the ASGI server. Environment variables for Neo4j credentials, Groq API key, and model configuration are injected through the Docker environment.
2. **Frontend container:** Serves the built React application through an Nginx reverse proxy that also forwards API requests to the backend container.
3. **Database:** Neo4j 5.23+ runs either as a container or as an external managed instance. The backend connects through the `NEO4J_URI` environment variable, supporting both `bolt://` and `neo4j+s://` schemes.

A Docker Compose file orchestrates the multi-container setup, defining service dependencies (the backend waits for Neo4j to be healthy before starting), port mappings (8000 for the API, 3000 for the frontend, 7687 for Neo4j Bolt), and volume mounts for persistent graph data storage. For development, the Vite dev server proxies API requests to the backend, eliminating the need for the Nginx layer.

5.2 Testing Framework

The backend includes a comprehensive test suite with 51 test cases organized across seven test modules:

Table 5.2: Test Suite Summary

Test Module	Tests	Coverage
test_pipeline.py	12	Pipeline orchestration, classification, fallback logic
test_retrievers.py	10	All three retriever strategies, retry mechanism
test_llm.py	8	GroqLLM invoke/ainvoke, error handling
test_api.py	8	API endpoints, request validation, CORS
test_integration.py	6	End-to-end pipeline flow with mocked services
test_config.py	4	Configuration loading, environment variable handling
conftest.py	3	Shared fixtures, mock factories
Total	51	

5.3 Evaluation Results

5.3.1 Query Classification Accuracy

The heuristic classifier was evaluated against a set of 40 standard test questions plus 15 novel questions:

Table 5.3: Query Classification Results

Query Type	Correct Route	Total	Accuracy
Structured	18	20	90%
Semantic	12	15	80%
Hybrid / Exploratory	17	20	85%
Overall	47	55	85.5%

5.3.2 Retrieval Quality by Strategy

Each retriever was evaluated independently on questions belonging to its target domain:

Table 5.4: Per-Retriever Success Rates

Retriever	1st Attempt	With Retry	With Fallback	Notes
Text2Cypher	82%	95%	98%	Self-healing recovers most syntax and property errors
Vector	90%	—	95%	No retry mechanism; fallback to VectorCypher
VectorCypher	88%	—	93%	Fallback to Text2Cypher for unembedded entities

Text2Cypher shows the largest gap between first-attempt and retry success rates, confirming the value of the self-healing mechanism. The Vector Retriever achieves the highest first-attempt success rate on its target queries (short keyword lookups

and vague descriptions) because it does not depend on correct Cypher generation. VectorCypher occasionally falls short on queries where the relevant entities are not Crime nodes—since only Crime nodes carry embeddings, questions about persons or officers that do not mention a specific crime type may miss relevant results until the fallback to Text2Cypher provides a direct structural match.

5.3.3 Response Time Performance

Table 5.5: Average Response Times by Module

Module	Average Time	Success Rate
Module 1: Query Classification	200–800 ms	85–95% first attempt
Module 2: Retrieval + Execution	50–500 ms	95%+ with retry
Module 3: Answer Generation	200–600 ms	99%+
Module 4: Visualization Rendering	50–200 ms	100%
Total Pipeline	1–3 seconds	95%+ overall

5.3.4 Comparative Evaluation

Table 5.6 compares Investigraph against the legacy NL-to-Cypher system that preceded it:

Table 5.6: Comparison with Legacy System

Feature	Legacy System	Investigraph (GraphRAG)
Pipeline	NL $\rightarrow$ Cypher $\rightarrow$ Results	NL $\rightarrow$ Routing $\rightarrow$ Multi-Strategy $\rightarrow$ RAG
Retrieval modes	Structured only	Structured + Semantic + Hybrid
Error handling	Basic single retry	Self-healing with 3 attempts + fallback
Context enrichment	None (flat results)	2-hop graph neighborhood traversal
LLM calls per query	4 (parse + plan + generate + answer)	1–2 (generate + answer)
Schema context	Narrow (2 node types)	Full schema always available
Few-shot examples	None	40 curated patterns
Visualization	None	Interactive graph (vis-network)

5.4 Challenges and Limitations

1. **Heuristic classifier boundaries.** The keyword-based classifier occasionally misroutes ambiguous queries, such as questions that combine structured elements with semantic phrasing. An LLM-based classifier would improve accuracy but at the cost of additional latency.
2. **Embedding coverage.** Only Crime nodes are embedded; other node types (Person, Location, Officer) do not have vector representations. This limits the scope of semantic search to crime-related queries.
3. **LLM dependency.** The system’s quality is bounded by the LLM’s capability. While Groq Llama-3.3-70b performs well on the 40-example prompt, more complex queries occasionally produce incorrect Cypher that even the self-healing mechanism cannot correct within three attempts.
4. **Data freshness.** The embedding vectors are generated once during setup. If new crimes are added to the database, the embedding script must be re-run to include them in the vector index.

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 Query Routing Effectiveness

The heuristic question classifier demonstrates that a lightweight, rule-based approach can achieve effective routing without the overhead of an additional LLM call. The 85.5% classification accuracy across all query types is attributable to the cascaded priority structure, where more specific rules (explicit keywords, area codes) fire before more general ones (semantic patterns, default routing). The misclassified queries—approximately 15%—typically involve questions that blend structured and semantic intent, such as “explain which areas have the most drug crimes,” where the word “explain” triggers the semantic route but the underlying question is best served by a structured aggregation.

6.2 Retrieval Strategy Effectiveness

Each retriever demonstrates clear strengths on its target query type:

- **Text2Cypher** achieves the highest precision on structured queries (counts, filters, aggregations), correctly answering 90%+ of such questions on the first attempt and reaching 95%+ with self-healing retries.
- **Vector Retriever** handles short keyword queries and vague descriptions effectively, leveraging the semantic proximity of embedding vectors to retrieve relevant crimes even when the exact terminology does not match the database records.
- **VectorCypher** produces the richest answers by combining semantic relevance with structural context. An answer from VectorCypher typically includes not just the crime details but also the names of involved persons, the investigating officers, and the geographic area—information that neither Text2Cypher nor plain Vector search automatically provides.

6.3 Self-Healing Success Rate

The self-healing mechanism recovers approximately 60–70% of initially failing Text2Cypher queries. The most common recovery scenarios involve:

- Correcting property name capitalization or casing mismatches (e.g., `Crime.Type` vs. `Crime.type`).
- Replacing invalid property values with valid alternatives from the schema’s distinct value list.
- Adjusting relationship directions (e.g., correcting a bidirectional match to a directed traversal).
- Relaxing overly specific filters—for example, replacing an exact match such as `c.type='Theft'` with a case-insensitive check like `toLower(c.type) CONTAINS 'theft'`.

Queries that fail all three attempts typically involve fundamentally incorrect structural assumptions that the LLM cannot resolve without additional context.

6.4 End-to-End Response Times

Typical response times for the complete pipeline range from 1 to 3 seconds, dominated by the LLM inference call:

- Simple count queries (single-hop, no retry): under 1 second.
- Relationship queries (single-hop traversal): 1–2 seconds.
- Multi-hop queries (3+ relationship hops): 2–3 seconds.
- Complex aggregations with retry: 3–5 seconds.

The Groq LPU architecture contributes significantly to these response times, with LLM calls typically completing in under 500 ms.

6.5 Scenario-Based Observations

Scenario 1: Structured count query. *Question:* “How many crimes happened in area WN?” The classifier correctly routes to Text2Cypher. The generated Cypher performs a 3-hop traversal (`Crime → Location → Area`) with a filter on the area code. The result is returned in under 1 second.

Scenario 2: Person-specific lookup. *Question:* “What crimes is Raymond Walker involved in?” The classifier routes to Text2Cypher based on the “involved in” pattern. The generated Cypher matches the person by name and traverses the `PARTY_TO` relationship to find associated crimes. Previous iterations of the system

failed on this query because the vector retriever could not surface person-specific results, validating the multi-strategy approach.

Scenario 3: Exploratory investigation. *Question:* “Tell me about incidents involving theft or robbery.” The classifier routes to VectorCypher. Semantic search identifies crimes semantically related to theft and robbery, and the graph traversal enriches each result with connected persons and officers. The investigator receives a comprehensive overview without needing to formulate a precise Cypher query.

Scenario 4: Self-healing recovery. *Question:* “Find all drug-related crimes in BL6 area that are under investigation.” The initial Cypher query uses an incorrect label (`:AREA` instead of `:Area`). The self-healing mechanism captures the Neo4j warning, feeds it back to the LLM, and the corrected query succeeds on the second attempt.

Scenario 5: Network discovery through relationship traversal. *Question:* “Who are the family members of people involved in drug offences?” The classifier routes to Text2Cypher based on the “involved in” pattern combined with the relationship keyword “family members.” The generated Cypher performs a 3-hop traversal: first matching Crime nodes where `type` contains “Drugs,” then traversing `PARTY_TO` to find connected persons, and finally following `FAMILY_REL` edges to discover their family members. The query returns a list of family connections that would be extremely difficult to surface through manual database browsing. This scenario demonstrates the value of the multi-hop traversal capability for uncovering indirect associations in criminal networks.

Scenario 6: Fallback chain activation. *Question:* “Recent incidents near Bolton.” The classifier routes to Text2Cypher because of the implied geographic filter. However, the generated Cypher attempts to filter by a date range using a property format that does not match the stored data, returning an empty result set. The self-healing mechanism attempts three reformulations—first relaxing the date filter, then adjusting the area matching—but all three attempts return empty results because “Bolton” is not stored as a direct property value in the graph (the data uses area codes such as “BL” instead of city names). The fallback chain then activates, routing the query to VectorCypher. The semantic search component successfully identifies crimes whose text descriptions mention the Bolton area, and the graph traversal enriches these results with connected persons, officers, and locations. This scenario illustrates the critical importance of the fallback mechanism: without it, the investigator would have received no answer at all.

Scenario 7: Short keyword lookup. *Question:* “Shoplifting.” The classifier identifies a short query (single word, ≤ 4 words) and routes to the Vector Retriever. The embedding model encodes the term and retrieves the five most semantically similar Crime nodes from the vector index. The results include not only crimes

labeled “Shoplifting” but also related entries categorized as “Theft from the Person” and “Other Theft,” demonstrating the semantic generalization capability of the embedding-based approach. The response time for this query is under 800 ms, making it the fastest of the scenarios tested.

6.6 Key Findings

1. Multi-strategy retrieval substantially broadens the range of questions the system can handle compared to single-strategy approaches.
2. Self-healing retry logic increases the effective success rate from approximately 85% (first attempt) to above 95% (with retries).
3. VectorCypher produces the most investigatively useful answers because it automatically enriches results with neighborhood context.
4. Heuristic classification, while not perfect, provides an acceptable quality–latency trade-off for real-time investigative use.
5. The 40 few-shot examples are the single most impactful component for Text2Cypher quality; each new example has a measurable positive effect on query accuracy.

6.7 Practical Applications

Investigraph addresses several practical use cases in law enforcement:

- **Rapid intelligence briefings:** Investigators can query the knowledge graph in natural English to prepare briefings on specific areas, crime types, or persons of interest.
- **Network discovery:** The VectorCypher retriever surfaces connections between people, locations, and crimes that might not be apparent from tabular reports.
- **Crime pattern analysis:** Aggregation queries identify hotspots, repeat offenders, and temporal patterns across the dataset.
- **Evidence auditing:** The transparent display of generated Cypher queries and retriever metadata enables supervisors to verify the evidentiary basis of any analytical conclusion.

CHAPTER 7

CONCLUSION

This project has presented Investigraph, a multi-strategy GraphRAG framework for natural language querying of POLE crime investigation knowledge graphs. The system addresses the fundamental limitations of existing NL-to-graph-query systems—single-strategy retrieval, fragile error handling, context poverty, and hallucination risk—through a unified architecture that integrates three complementary retrieval strategies with intelligent routing, self-healing query execution, and context-grounded answer generation.

The principal contributions of this work are as follows:

1. **Multi-strategy retrieval architecture.** The combination of Text2Cypher, Vector, and VectorCypher retrievers within a single pipeline allows the system to handle a broader range of investigator questions than any single-strategy approach. The intelligent routing mechanism directs each question to the most appropriate strategy, while the circular fallback chain ensures that the system always has an alternative path when the primary strategy fails.
2. **Self-healing query execution.** The iterative error correction mechanism represents a practical solution to the fragility of LLM-generated Cypher queries. By feeding Neo4j error messages back to the model and supporting up to three correction attempts, the system recovers 60–70% of initially failing queries, raising the effective success rate from approximately 85% to above 95%.
3. **Context-enriched answers through hybrid retrieval.** The VectorCypher hybrid retriever demonstrates that combining semantic vector search with graph traversal produces substantially richer investigative context than either approach alone. Investigators receive not just matching crime records but the complete neighborhood of connected persons, officers, and locations.
4. **Transparent and auditable design.** Every response includes the generated Cypher query, the retriever selected, the context gathered, and the execution time. This transparency is essential for forensic applications where analytical conclusions must be traceable to specific data sources in the knowledge graph.
5. **Production-ready implementation.** The system is delivered as a complete, deployable application with a FastAPI backend, a React frontend featuring interactive graph visualization via vis-network, and comprehensive test coverage.

across 51 test cases. The 40 curated few-shot examples, cached schema introspection, and singleton-pattern resource management ensure efficient operation under real-time query loads.

The evaluation results confirm that Investigraph achieves response times of 1–3 seconds for the majority of queries, classification accuracy of 85.5% across structured, semantic, and hybrid query types, and an overall query success rate exceeding 95% when self-healing retries are enabled. These performance characteristics meet the requirements for interactive investigative use, where analysts expect near-instantaneous responses during time-sensitive operations.

The system’s reliance on the `neo4j-graphrag` library and Groq-hosted Llama-3.3-70b inference establishes a foundation that is both modular and upgradable: as more capable LLMs become available or as the `neo4j-graphrag` framework introduces additional retriever types, Investigraph can incorporate these advances with minimal architectural change. The POLE knowledge graph schema, widely adopted in law enforcement intelligence work, ensures that the system is directly applicable to real-world policing datasets once appropriate data governance controls are in place.

CHAPTER 8

FUTURE SCOPE

8.1 LLM-Based Query Routing

The current heuristic classifier could be replaced or augmented with an LLM-based classifier that analyzes the semantic intent of each question. While this would add an additional LLM call to the pipeline, it could improve classification accuracy on ambiguous queries that blend structured and semantic intent. A lightweight model such as a fine-tuned Phi-3-mini could provide classification in under 100 ms while capturing nuances that keyword matching misses.

One practical approach would be a two-tier system: the existing heuristic classifier handles clear-cut queries at zero additional cost, while a secondary LLM-based classifier is invoked only for questions that fall below a confidence threshold. This hybrid strategy would preserve the low-latency advantage of heuristics for the majority of queries while selectively applying more sophisticated reasoning to the ambiguous minority. Fine-tuning a small model on a labeled dataset of investigator questions, annotated with their ideal retrieval strategy, would further improve routing precision.

8.2 Multi-Modal Evidence Retrieval

Future versions could incorporate image embeddings for evidence photographs (crime scene images, CCTV captures) stored in the knowledge graph. By extending the vector index to support image embeddings alongside text embeddings, investigators could search for visually similar evidence across cases, opening new avenues for pattern detection.

Cross-modal retrieval models such as CLIP and its successors can project both textual descriptions and photographic evidence into a shared embedding space, enabling queries like “show me crime scenes that look similar to this photograph” or “find vehicles matching this CCTV capture.” Integrating these capabilities into the existing pipeline would require adding an image embedding property to relevant nodes (Crime, Vehicle, Object) and extending the vector index to support multiple embedding types. The VectorCypher retriever could then be generalized to combine visual similarity with graph traversal, producing results that link visual evidence to the persons, locations, and events connected through the POLE schema.

8.3 Temporal Crime Pattern Analysis

The current system treats each query independently. A temporal analysis module could detect trends and patterns over time—such as increasing crime rates in a specific area, seasonal variations in crime types, or the emergence of new criminal networks. Time-series analysis over the graph’s date properties would enable predictive intelligence capabilities.

Specific techniques that could be incorporated include sliding-window aggregation over crime dates to detect hotspot emergence, change-point detection algorithms to identify sudden shifts in crime patterns within geographic areas, and recurrence analysis to find periodic patterns (e.g., weekend spikes in certain crime categories). These temporal features could be exposed through natural language queries such as “how have drug crimes in area BL changed over the past six months” or “which areas are seeing an increase in burglaries,” with the system automatically generating time-series visualizations alongside the textual answer.

8.4 Advanced Graph Analytics

Integration of graph algorithms such as community detection (Louvain, Label Propagation), centrality analysis (betweenness, PageRank), and pathfinding (shortest paths between suspects) would transform Investigraph from a question-answering system into a comprehensive graph analytics platform. The Neo4j Graph Data Science (GDS) library provides production-ready implementations of these algorithms.

In particular, community detection could automatically identify clusters of individuals who are densely connected through shared crimes, family relationships, and communication records—effectively surfacing criminal organizations without requiring an analyst to formulate the right query. Centrality measures could rank individuals by their structural importance within the network, highlighting key intermediaries whose removal would most disrupt criminal communication channels. These analytics results could be cached as node properties and made queryable through the existing natural language interface, allowing investigators to ask questions like “who are the most connected people in the Bolton drug network” and receive graph-algorithm-backed answers.

8.5 Security and Privacy Enhancements

A production deployment for law enforcement would require role-based access control (RBAC) to restrict query scope based on the investigator’s clearance level, audit logging of all queries and results for compliance, and encryption of data at rest

and in transit. Integration with existing identity providers (LDAP, Active Directory) and compliance with data protection regulations such as GDPR would be necessary for deployment in European jurisdictions.

At the technical level, RBAC could be implemented through Neo4j's built-in role and privilege system, which supports fine-grained access control at the label and property level. This would allow a junior analyst to query crime statistics without seeing witness identities, while a senior investigator could access the full graph. The audit logging system would need to capture not only the natural language question but also the generated Cypher query, the retriever selected, the results returned, and the timestamp and identity of the querying officer. This audit trail is essential for court proceedings where the provenance of analytical conclusions may be challenged.

8.6 Scalability and Real-Time Data Integration

The current architecture processes queries against a static snapshot of the knowledge graph. Future work could integrate real-time data ingestion from dispatch systems, CCTV feeds, and social media intelligence, enabling the graph to grow dynamically and the embeddings to be updated incrementally rather than requiring a full re-computation. Distributed Neo4j clusters and streaming ingestion pipelines (e.g., Apache Kafka) would support this evolution.

An incremental embedding update mechanism would be particularly valuable: as new Crime nodes are added to the graph, a background worker could compute their embeddings and update the vector index without taking the system offline. Combined with Neo4j's causal clustering for read replicas, this would allow the system to serve queries continuously while ingesting new data. For large-scale deployments covering multiple jurisdictions, a federated architecture could allow each regional database to maintain its own POLE graph while supporting cross-jurisdictional queries through a coordinating gateway layer.

APPENDIX A

CODE ATTACHMENTS

This appendix presents the core source code of the Investigraph system. The listings are organized by functional module, covering the GraphRAG pipeline orchestrator, the self-healing Text2Cypher retriever, the VectorCypher hybrid retrieval query, the GroqLLM wrapper, and the embedding singleton.

A.1 GraphRAG Pipeline Orchestrator

The `GraphRAGPipeline` class is the central orchestrator. It classifies each incoming question, routes it to the appropriate retriever, executes fallback logic when the primary retriever returns empty results, and generates a context-grounded answer.

```
1 class GraphRAGPipeline:
2     """Orchestrates GraphRAG retrieval and answer generation"""
3
4     def __init__(self):
5         self.settings = get_settings()
6         self._driver: Optional[Driver] = None
7         self._llm: Optional[LLMInterface] = None
8         self._text2cypher = None
9         self._vector_retriever = None
10        self._vector_cypher_retriever = None
11        self._rag_instances: Dict[str, GraphRAG] = {}
12        self._initialized = False
13
14    def initialize(self) -> None:
15        """Initialize pipeline - build all retrievers and RAG
16        instances"""
17
18        if self._initialized:
19            return
20
21        embedder = get_embedder()
22
23        self._text2cypher = build_text2cypher_retriever(
24            driver=self.driver, llm=self.llm,
25            neo4j_database=self.settings.NEO4J_DATABASE,
26        )
27
28        self._vector_retriever = build_vector_retriever(
29            driver=self.driver, embedder=embedder,
30            index_name=self.settings.VECTOR_INDEX_NAME,
```

```

27         neo4j_database=self.settings.NEO4J_DATABASE,
28     )
29     self._vector_cypher_retriever =
30     build_vector_cypher_retriever(
31         driver=self.driver, embedder=embedder,
32         index_name=self.settings.VECTOR_INDEX_NAME,
33         neo4j_database=self.settings.NEO4J_DATABASE,
34     )
35     rag_template = RagTemplate(
36         template=get_rag_answer_template(),
37         expected_inputs=["query_text", "context"],
38     )
39     for name, retriever in [
40         ("text2cypher", self._text2cypher),
41         ("vector", self._vector_retriever),
42         ("vector_cypher", self._vector_cypher_retriever),
43     ]:
44         if name == "text2cypher":
45             base_retriever = retriever._build_retriever()
46             self._rag_instances[name] = GraphRAG(
47                 retriever=base_retriever, llm=self.llm,
48                 prompt_template=rag_template,
49             )
50         else:
51             self._rag_instances[name] = GraphRAG(
52                 retriever=retriever, llm=self.llm,
53                 prompt_template=rag_template,
54             )
55     self._initialized = True
56
57     def run(self, question: str) -> Dict[str, Any]:
58         """Run GraphRAG pipeline with intelligent retriever
59         selection."""
60         start_time = time.time()
61         retriever_used = "unknown"
62         retriever_context, results = [], []
63         cypher, error = None, None
64         graph_data = {"nodes": [], "edges": []}
65
66         try:

```

```

66         retriever_name = self._classify_question(question)
67         retriever_used = retriever_name
68         answer, cypher, results, graph_data, retriever_context = \
69             self._execute_retriever(retriever_name, question)
70
71         if not answer or answer.strip() == "":
72             fallback = self._get_fallback(retriever_name)
73             if fallback:
74                 retriever_used = f"{retriever_name} -> {fallback}"
75
76         answer, cypher, results, graph_data, \
77         retriever_context = \
78             self._execute_retriever(fallback, question)
79         except Exception as e:
80             error = str(e)
81             answer = f"Unable to process query: {str(e)}"
82
83         elapsed_ms = int((time.time() - start_time) * 1000)
84         return {
85             "question": question, "answer": answer or "No results
86             found.",
87             "cypher": cypher or "", "results": results,
88             "graph_data": graph_data, "attempts": 1,
89             "execution_time_ms": elapsed_ms,
90             "retriever_used": retriever_used,
91             "retriever_context": retriever_context,
92         }

```

Listing A.1: GraphRAG Pipeline — Initialization and Run Method

A.2 Heuristic Question Classifier

The classifier uses a prioritized cascade of heuristic rules to route each question to the most appropriate retrieval strategy without requiring an additional LLM call.

```

1 def _classify_question(self, question: str) -> str:
2     """Classify question to select the best retriever."""
3     q = question.lower().strip()
4
5     # 1. Explicit structured patterns -> Text2Cypher
6     structured_keywords = [

```

```

7         "how many", "count", "total", "which area", "which officer"
8     ,
9         "top ", "most ", "least ", "average", "repeat offender",
10        "network", "linked", "relationship between",
11        "calls between", "phone", "investigated by",
12    ]
13    for pattern in structured_keywords:
14        if pattern in q:
15            return "text2cypher"
16
17    # 2. Filter-heavy queries -> Text2Cypher
18    import re
19    area_code_pattern = re.compile(
20        r'\b(?:in\s+)?(?:area\s+)?[A-Z]{1,3}\d{1,2}\b', re.
21        IGNORECASE
22    )
23    has_area_filter = bool(area_code_pattern.search(question))
24
25    outcome_keywords = [
26        "under investigation", "no suspect", "charged",
27        "cautioned", "awaiting", "unable to prosecute",
28    ]
29    has_outcome_filter = any(kw in q for kw in outcome_keywords)
30
31    entity_keywords = [
32        "people involved", "persons involved", "evidence",
33        "officers handling", "officers investigating",
34    ]
35    entity_hits = sum(1 for kw in entity_keywords if kw in q)
36    has_multi_entity = entity_hits >= 2
37
38    crime_type_keywords = [
39        "drug", "theft", "burglary", "robbery", "assault",
40        "violence", "criminal damage", "shoplifting", "fraud",
41    ]
42    has_crime_type = any(kw in q for kw in crime_type_keywords)
43
44    filter_score = sum([
45        has_area_filter, has_outcome_filter, has_multi_entity,
46        has_crime_type and (has_area_filter or has_outcome_filter)
47    ],

```

```

45     ])
46     if filter_score >= 2:
47         return "text2cypher"
48
49     # 3. Person-specific queries -> Text2Cypher
50     person_patterns = [
51         "involved in", "connected to", "associated with",
52         "crimes of", "crimes committed by", "family of",
53     ]
54     if any(p in q for p in person_patterns):
55         return "text2cypher"
56
57     # 4. Semantic / exploratory -> VectorCypher
58     semantic_patterns = [
59         "tell me about", "describe", "explain", "summarize",
60         "what do you know", "overview", "similar to",
61     ]
62     for pattern in semantic_patterns:
63         if pattern in q:
64             return "vector_cypher"
65
66     # 5. Question starters -> Text2Cypher
67     if q.startswith(("who ", "what ", "where ", "when ")):
68         return "text2cypher"
69
70     # 6. Short queries -> Vector (keyword lookup)
71     if len(q.split()) <= 4:
72         return "vector"
73
74     # 7. Default -> VectorCypher (most robust general strategy)
75     return "vector_cypher"

```

Listing A.2: Question Classifier — Heuristic Routing Logic

A.3 Self-Healing Text2Cypher Retriever

The `Text2CypherRetrieverWithRetry` class wraps the standard `Text2CypherRetriever` from the `neo4j-graphrag` library with an iterative error-correction loop that feeds syntax errors and empty results back to the LLM for self-healing.

```

1 MAX_CYPHER_RETRIES = 3
2
3 class Text2CypherRetrieverWithRetry:

```

```

4      """Wraps Text2CypherRetriever with self-healing retry logic."""
5
6      def __init__(self, driver, llm, neo4j_schema, examples,
7                  neo4j_database="pole", custom_prompt=""):
8          self.driver = driver
9          self.llm = llm
10         self.neo4j_schema = neo4j_schema
11         self.examples = examples
12         self.neo4j_database = neo4j_database
13         self.custom_prompt = custom_prompt
14
15     def _build_retriever(self, extra_instructions=""):
16         """Build a fresh retriever with optional error context."""
17         prompt = self.custom_prompt
18         if extra_instructions:
19             prompt += (
20                 f"\n\nERROR CONTEXT\n{extra_instructions}\n"
21                 "Please fix the query based on the error above.\n"
22             )
23         return Text2CypherRetriever(
24             driver=self.driver, llm=self.llm,
25             neo4j_schema=prompt + "\n\n" + self.neo4j_schema,
26             examples=self.examples,
27             neo4j_database=self.neo4j_database,
28         )
29
30     def search(self, query_text, **kwargs):
31         """Search with retry on Cypher errors and empty results."""
32         last_error = None
33
34         for attempt in range(1, MAX_CYPHER_RETRIES + 1):
35             try:
36                 if attempt == 1:
37                     retriever = self._build_retriever()
38                 else:
39                     retriever = self._build_retriever(
40                         extra_instructions=last_error
41                     )
42                 result = retriever.search(
43                     query_text=query_text, **kwargs

```

```

44         )
45
46         # Check for empty results
47         if not result.items:
48             last_error = (
49                 "Previous query returned 0 results. Try:\n"
50                 "- Relaxing WHERE filters\n"
51                 "- Using CONTAINS instead of exact matches\n"
52                 "- Checking relationship direction\n"
53             )
54             if attempt < MAX_CYPHER_RETRIES:
55                 continue
56             return result
57
58         except Exception as e:
59             last_error = f"Query failed: {str(e)}"
60             if attempt >= MAX_CYPHER_RETRIES:
61                 raise
62
63         raise RuntimeError("Text2Cypher max retries exceeded")

```

Listing A.3: Text2Cypher with Self-Healing Retry

A.4 VectorCypher Hybrid Retrieval Query

The VectorCypher retriever combines semantic vector search with a Cypher graph traversal that gathers the 2-hop neighborhood of each retrieved crime node, including connected persons, investigating officers, and geographic information.

```

1 def build_vector_cypher_retriever(driver, embedder,
2     index_name="crime_vector_index", neo4j_database="pale"):
3     """Build VectorCypherRetriever that enriches vector results
4     with graph traversal context."""
5
6     retrieval_query = """
7         WITH node AS crime, score
8         OPTIONAL MATCH (p:Person)-[:PARTY_TO]->(crime)
9         OPTIONAL MATCH (crime)-[:OCCURRED_AT]->(l:Location)
10        OPTIONAL MATCH (l)-[:LOCATION_IN_AREA]->(a:Area)
11        OPTIONAL MATCH (crime)-[:INVESTIGATED_BY]->(o:Officer)
12        RETURN crime.id AS crime_id,
13                crime.type AS crime_type,
14                crime.date AS crime_date,

```

```

15         crime.last_outcome AS outcome,
16         collect (DISTINCT p.name + ' '
17             + coalesce(p.surname, '')) AS people_involved,
18         l.address AS location,
19         l.postcode AS postcode,
20         a.areaCode AS area,
21         collect (DISTINCT o.name + ' ('
22             + coalesce(o.rank, '') + ')') AS officers,
23         score
24     """
25
26     def result_formatter(record):
27         people = [p for p in record.get("people_involved", [])
28             if p and p.strip()]
29         officers = [o for o in record.get("officers", [])
30             if o and o.strip()]
31
32         content_parts = [
33             f"Crime: {record.get('crime_type', 'Unknown')} ",
34             f"(ID: {record.get('crime_id', 'N/A')})",
35             f"Date: {record.get('crime_date', 'Unknown')}",
36             f"Outcome: {record.get('outcome', 'Unknown')}",
37         ]
38         if record.get("location"):
39             content_parts.append(
40                 f"Location: {record['location']}, "
41                 f"{record.get('postcode', '')}"
42             )
43         if record.get("area"):
44             content_parts.append(f"Area: {record['area']}")
45         if people:
46             content_parts.append(
47                 f"People involved: {' , '.join(people)}"
48             )
49         if officers:
50             content_parts.append(
51                 f"Officers: {' , '.join(officers)}"
52             )
53
54         return RetrieverResultItem(
55             content="\n".join(content_parts),

```

```

56         metadata={
57             "crime_id": record.get("crime_id"),
58             "crime_type": record.get("crime_type"),
59             "area": record.get("area"),
60             "score": record.get("score"),
61         },
62     )
63
64     return VectorCypherRetriever(
65         driver=driver, index_name=index_name,
66         retrieval_query=retrieval_query,
67         result_formatter=result_formatter,
68         embedder=embedder, neo4j_database=neo4j_database,
69     )

```

Listing A.4: VectorCypher Retriever with Graph Traversal

A.5 GroqLLM Wrapper

The GroqLLM class implements the LLMInterface from the neo4j-graphrag library, connecting the pipeline to Groq's hosted Llama-3.3-70b model with both synchronous and asynchronous invocation methods.

```

1  class GroqLLM(LLMInterface):
2      """Groq LLM implementing neo4j-graphrag's LLMInterface"""
3
4      def __init__(self, model_name="llama-3.3-70b-versatile",
5                   model_params=None):
6          settings = get_settings()
7          self.client = Groq(api_key=settings.GROQ_API_KEY)
8          self.model_name = model_name
9          self.model_params = model_params or {}
10
11     def invoke(self, input, message_history=None,
12               system_instruction=None):
13         """Synchronous LLM invocation."""
14         temperature = self.model_params.get("temperature", 0)
15         max_tokens = self.model_params.get("max_tokens", 4096)
16
17         messages = []
18         if system_instruction:
19             messages.append({
20                 "role": "system",

```

```

21         "content": system_instruction,
22     })
23     if message_history:
24         if hasattr(message_history, 'messages'):
25             history_list = message_history.messages
26         else:
27             history_list = message_history
28         for msg in history_list:
29             if isinstance(msg, dict):
30                 messages.append(msg)
31             else:
32                 messages.append({
33                     "role": msg.get("role", "user"),
34                     "content": msg.get("content", ""),
35                 })
36
37     messages.append({"role": "user", "content": input})
38
39     response = self.client.chat.completions.create(
40         model=self.model_name, messages=messages,
41         temperature=temperature, max_tokens=max_tokens,
42     )
43     content = response.choices[0].message.content
44     return LLMResponse(content=content)
45
46     async def ainvoke(self, input, message_history=None,
47                       system_instruction=None):
48         """Async invocation (delegates to sync for Groq SDK)."""
49         return self.invoke(
50             input, message_history=message_history,
51             system_instruction=system_instruction,
52         )

```

Listing A.5: GroqLLM — neo4j-graphrag Compatible LLM Wrapper

A.6 Embedding Pipeline

The embedding module initializes the SentenceTransformer model as a singleton and provides it to both the Vector and VectorCypher retrievers.

```

1 from neo4j_graphrag.embeddings import
    SentenceTransformerEmbeddings
2 from app.config import get_settings

```

```

3
4 _embedder = None
5
6 def get_embedder():
7     """Get or create SentenceTransformer embedder singleton"""
8     global _embedder
9     if _embedder is None:
10         settings = get_settings()
11         _embedder = SentenceTransformerEmbeddings(
12             model=settings.EMBEDDING_MODEL # all-MiniLM-L6-v2
13         )
14     return _embedder

```

Listing A.6: SentenceTransformer Embedder Singleton

A.7 FastAPI Application Entry Point

The main application module configures the FastAPI server, initializes the GraphRAG pipeline at startup, and exposes the query endpoint.

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     """Lifecycle manager for FastAPI application"""
4     # Startup
5     neo4j_client = get_neo4j_client()
6     pipeline = get_graphrag_pipeline()
7     pipeline.initialize()
8     case_studies = get_case_study_loader().load()
9     yield
10    # Shutdown
11    pipeline.close()
12    neo4j_client.close()
13
14 app = FastAPI(
15     title="Investigraph GraphRAG API",
16     description="GraphRAG-powered query API for POLE "
17         "crime investigation knowledge graph",
18     version="2.0.0", lifespan=lifespan,
19 )
20
21 @app.post("/api/query", response_model=QueryResponse)
22 async def query(request: QueryRequest):
23     """Natural language query endpoint"""

```

```

24     pipeline = get_graphrag_pipeline()
25     result = pipeline.run(request.question)
26     return QueryResponse(
27         question=result["question"],
28         answer=result["answer"],
29         cypher=result.get("cypher", ""),
30         results=result["results"],
31         graph_data=GraphData(**result["graph_data"]),
32         attempts=result["attempts"],
33         execution_time_ms=result["execution_time_ms"],
34         retriever_used=result.get("retriever_used"),
35         retriever_context=result.get("retriever_context", []),
36     )

```

Listing A.7: FastAPI Application with Lifecycle Management

BIBLIOGRAPHY

- [1] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. 2nd. O'Reilly Media, 2015.
- [2] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. “Cypher: An Evolving Query Language for Property Graphs”. In: *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. ACM. 2018, pp. 1433–1445. DOI: 10.1145/3183713.3190657.
- [3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. “Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation”. In: *Proceedings of the VLDB Endowment* 17.5 (2024), pp. 1132–1145.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 9459–9474.
- [5] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. “Retrieval-Augmented Generation with Graphs (GraphRAG)”. In: *arXiv preprint arXiv:2501.00309* (2025).
- [6] Victor Zhong, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning”. In: *arXiv preprint arXiv:1709.00103* (2017).
- [7] Groq, Inc. *Groq: High-Speed LLM Inference Platform*. <https://groq.com/>. Accessed: April 2026. 2024.
- [8] Neo4j, Inc. *neo4j-graphrag-python: Neo4j GraphRAG Package for Python*. <https://github.com/neo4j/neo4j-graphrag-python>. Accessed: April 2026. 2024.

- [9] David Robinson and Chris Scogings. “The Detection of Criminal Groups in Real-World Fused Data: Using the Graph-Mining Algorithm “GraphExtract””. In: *Security Informatics* 7.2 (2018), pp. 1–14. DOI: 10.1186/s13388-018-0031-9.
- [10] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2018, pp. 3911–3921.
- [11] Malcolm K Sparrow. “The Application of Network Analysis to Criminal Intelligence: An Assessment of the Prospects”. In: *Social Networks* 13.3 (1991), pp. 251–274.
- [12] Valdis E Krebs. “Mapping Networks of Terrorist Cells”. In: *Connections* 24.3 (2002), pp. 43–52.
- [13] Hsinchun Chen, Wingyan Chung, Jennifer J Xu, Gang Wang, Yi Qin, and Michael Chau. “Crime Data Mining: A General Framework and Some Examples”. In: *Computer* 37.4 (2004), pp. 50–56.
- [14] Siddharth Kaza and Hsinchun Chen. “Social Network Analysis of Criminal Networks”. In: *IEEE Intelligent Systems* 24.5 (2009), pp. 71–73.
- [15] Wei Chen, Fangzhou Guo, and Fei-Yue Wang. “A Survey of Visual Analytics Techniques and Applications: State-of-the-Art Research and Future Challenges”. In: *Journal of Computer Science and Technology* 31.5 (2016), pp. 852–867.
- [16] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. “Foundations of Modern Query Languages for Graph Databases”. In: *ACM Computing Surveys* 50.5 (2017), pp. 1–40.
- [17] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37.
- [18] Maximilian T Fischer, Yannick Metz, Lucas Joos, Matthias Miller, and Daniel A Keim. “MULTI-CASE: A Transformer-based Ethics-aware Multimodal Investigative Intelligence Framework”. In: *arXiv preprint arXiv:2401.01955* (2024).

- [19] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), pp. 1877–1901.
- [20] Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. “Robust Text-to-SQL Generation with Execution-Guided Decoding”. In: *arXiv preprint arXiv:1807.03100* (2018).
- [21] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. “Synchromesh: Reliable Code Generation from Pre-Trained Language Models”. In: *arXiv preprint arXiv:2201.11227* (2022).
- [22] Francesco Cazzaro, Justin Kleindienst, Sofia Marquez Gomez, and Ariadna Quattoni. “ZOGRASCOPE: A New Benchmark for Semantic Parsing over Property Graphs”. In: *arXiv preprint arXiv:2503.05268* (2025).
- [23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023).
- [24] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019, pp. 3982–3992.
- [25] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: (2019), pp. 4171–4186.
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)* 30 (2017).
- [27] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-Scale Similarity Search with GPUs”. In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547.
- [28] Sebastian Ramirez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. <https://fastapi.tiangolo.com/>. Accessed: April 2026. 2024.

- [29] vis.js Community. *vis-network: A Dynamic, Browser-Based Network Visualization Library*. <https://visjs.github.io/vis-network/>. Accessed: April 2026. 2024.